```
// POINTER1.CPP
// THIS IS THE FIRST PROGRAM FOR THE POINTER
// IT EXPLAIN THE WHAT IS CONTENT AND WHAT IS VALUE OF THE VARIABLES.

#include <stdio.h>
#include <conio.h>

void main()
{
  clrscr();
  int i=10,j=20,k=30;
  char ch1='C',ch2='Z';
  float ans=5.253;

  printf("The value of I is %d stored at %u",i,&i);
  printf("\nThe value of J is %d stored at %u",j,&j);
  printf("\nThe value of K is %d stored at %u",k,&k);
  printf("\nThe value of ch1 is %c stored at %u",ch1,&ch1);
  printf("\nThe value of ch2 is %c stored at %u",ch2,&ch2);
  printf("\nThe value of ans is %f stored at %u",ans,&ans);

  *&i   = *&i   * 10;
  *&ch1 = *&ch1 + 1;

  printf("\nThe value of I is %d stored at %u",*&i,&i);
  printf("\nThe value of ch1 is %c stored at %u",*&ch1,&ch1);
  *&ans = 10.235;

  printf("\nThe value of ans is %f stored at %u",*&ans,&ans);
  getch();

}
//----------------------O U T - P U T---------------------------------
```

```
// POINTER2.CPP ==> POINTER TO UNDERSTAND THE ADDRESS AND CONTENT
// Value   ==> Address of Variable store at Pointer Variable
// Content ==> Content of Variable where Pointer is Pointing
// Store   ==> Address of Variable or Pointer Variable

#include <stdio.h>
#include <conio.h>

void main()
   {
     int i;
     int *intpt;
     i=10;

     clrscr();
     intpt=&i;    //  intpt = i   cannot convert int to int * error display

     printf("\nThe Content of i     is %5d and stored at %5u",i,&i);
     printf("\nThe Value of intpt   is %5u and stored at %5u",intpt,&intpt);
     printf("\nThe Content of intpt is %5d and stored at %5u",*intpt,intpt);

     i=20;               //CHANGE i VALUE DIRECTLY
     printf("\nThe Content of intpt   is %5d and stored at %5u",*intpt,intpt);
     *intpt = *intpt + 10;  // CHANGE i VALUE INDIRECTLY
     printf("\nThe Content of intpt is %5d and stored at %5u",*intpt,intpt);
         getch();
       }

//       i                  intptr
//  +----------+        +----------+
//  |    10    |        |  65524 |
//  +----------+        +----------+
//     65524              65522
//
//----------------------O U T   -   P U T-----------------------------
```

```
// POINTER3.CPP
// SCAN THE STRING WITH THE USE OF POINTER AND DISPLAY IT ON SCREEN
// WHEN EVER NULL VALUE IS PLACED THEN ADDRESSING STRAT FROM 0
// OTHERWISE INITILA ADDRESS ARE THERE IN PTR

#include <stdio.h>
#include <conio.h>
#include <string.h>

void main()
   {
     clrscr();
     int *ptr;
     printf("The value of ptr %u stored at %u\n",ptr,&ptr);
     ptr = NULL;   // REMOVE NULL VALUE AND WHAT HAPPEND CHECK

     printf("The value of ptr %u stored at %u\n",ptr,&ptr);
     int i,n;
     printf("How many values are there?  ");
     scanf("%d",&n);

     printf("Initial value of ptr  is  : %u and address is %u\n",ptr,&ptr);
     for (i=0;i<n;i++)
          {
               printf("Enter ptr value [%d] and address is %u : ",i,(ptr+i));
               scanf("%d",(ptr+i)); // DO NOT WRITE & SIGN
          }

     printf("\nDisplay the value on Screen for ptr\n");

     for (i=0;i<n;++i)
          {
               printf("The value is : %d and the address is %u\n",*(ptr+i),ptr+i);
          }

         getch();
       }
// --------------------------O U T P U T-----------------------------------
```

```
// POINTER5.CPP
// HOW ONE DIMENTIONAL CHARACTER POINTER WORK AND DISPLAY THE
CONTENT
// WITH ADDRESS ON SCREEN
// THIS IS THE THIRD PROGRAM FOR POINTER


#include <stdio.h>
#include <conio.h>

void main()
        {
          clrscr();       char *ptr;            int i;
          printf("The value of ptr is %u and stroed at %u\n",ptr,&ptr);

          printf("Enter the string: ");     gets(ptr);

          printf("The string is:  ");         puts(ptr);

          printf("The content of the ptr is %c and address is %u\n",*ptr,ptr);

          for (;*ptr!=NULL;)
                {
                  printf("content is %-2c",*ptr);
                  printf("Address %u\n",ptr);
                  ptr++;
                }
          getch();
        }
```

```
// POINTER6.CPP
// TO STORE THE 5 INTEGER VALUE IN POINTER AND DISPLAY ON SCREEN
// DISPLAY ON REVERSE ORDER INWHICH THEY ARE INSERTED
#include <stdio.h>
#include <conio.h>

void main()
  {
        clrscr();       int *ptr;       int i;
        printf("Initial value of ptr is : %u store at %u\n",ptr,&ptr);

        for (i=0;i<5;i++)
              {
               printf("Enter the value at %u:  ",ptr);
               scanf("%d",ptr); // Equal scanf("%d",&*ptr)
               ptr++;
              }

        for (i=0;i<5;i++)
              {
               ptr--;
               printf("\nThe value is : %d and the address %u",*ptr,ptr);
              }

              getch();
       }
//--------------O U T   P U T----------------------------------------
```

```
// PTLIBFUN.CPP
// THIS PROGRAM USE FOR STRLEN,STRCPY,STRCAT,STRCMP FUNCTIONS
WITH POINTER.
#include <stdio.h>
#include <conio.h>
#include <string.h>
void main()
{
 clrscr();
 char *ptr1={"Bhavnagar "},*ptr2={"University"},*ptr3; //
INITIALIZATION
 printf("Starting Address of ptr1 is %u\n",ptr1);
 printf("Starting Address of ptr2 is %u\n",ptr2);
 printf("Starting Address of ptr3 is %u\n",ptr3);

 printf("Length of ptr1 is %d store at %u\n",strlen(ptr1),ptr1);
 // int a[10] strlen(a) i.e address of one-dimentioal array
 printf("Length of ptr2 is %d store at %u\n",strlen(ptr2),ptr2);

 strcpy(ptr3,ptr2);
 puts(ptr3);
 printf("String concatination\n");
 strcat(ptr1,"Ram");
 puts(ptr1);   //OVERWRITE THE CONTENT
 printf("\n");

 strcat(ptr3,strcat(ptr1,ptr2));
 puts(ptr1);
 puts(ptr3);

 if (strcmp(ptr1,ptr3) < 0)
        printf("ptr3 string is grater than the ptr1 string\n");

 getch();

}

//-------------------O U T    P U T-----------------------------------
```

```
// PTMAXFN.CPP
// THIS PROGRAM FIND OUT THE MAXIMUM VALUE FROM THE ARRAY
WITH  USE OF POINTER.
#include <stdio.h>   #include <conio.h>
void main()
 {
        float *ptr,a[6] = {6.2,62.23,1.0,0.0,-55.24,21.09},max;        int i=0;
        ptr=&a[0];    max=*ptr;          clrscr();

        printf("Base Address of a is %u\n",a);
        printf("Base Address of a is %u\n",&a[0]);

        for (i=0;i<6;i++)
              {
               printf("Content of a[%d] =  %-9.2f",i,*ptr);
               printf("Address %6u\n",ptr++);
              }

        printf("Address of a[0] = %u and ptr = %u\n",&a[0],ptr);
        ptr=a;  // ptr = &a[0]   BOTH ARE SAME

        printf("Address of a[0] = %u and ptr = %u\n",&a[0],ptr);
        i=0;
        while (i < 6)
         {
              if ( max < *ptr )
                    {
                      max = *ptr;
                    }
              i++;

              ptr++;

             }

        printf("Maximum is %f",max);

        getch();

 }
```

```
// PTREVCHA.CPP
// THIS PROGRAM REVERSE THE CONTENT OF THE CHARACTER ARRAY OF
THE STRING
// WITH USE OF POINTER ONLY.

#include <stdio.h>
#include <conio.h>
#include <string.h>

void main()
        {
          char *ptr,temp;    int n,i=0;
        clrscr();
        printf("Enter the string :  ");     gets(ptr);
          printf("The base address is %u store at %u\n",ptr,&ptr);

        while (*(ptr+i) !=NULL)
                {
                  printf("Content is %2c address %u\n",*(ptr+i),(ptr+i));
                  i++;
                }
          n = strlen(ptr);     i = 1;
          do
                {
                  temp = *(ptr+n-i);
                  *(ptr+n-i) = *(ptr+i-1);

                  *(ptr+i-1) = temp;

                  i++;

                } while (n/2 >= i);

          puts(ptr);

}
```

```
// This program shows the that Array and Pointer works in a same manner
        #include <stdio.h>
        #include <conio.h>
        void main()
           {
            clrscr();
            int *ptr, a[5]= {10,20,30,40,50}, i;
            ptr = a;
            printf("Display the array Elements\n");
            for (i=0;i<5;i++)
                {
                  printf("The value of a[%d] = %5d stored at %5u\n",i,a[i],&a[i]);
                }
            printf("Display the array Elements With Pointers\n");
            for (i=0;i<5;i++)
                {
                  printf("The value of a[%d] = %5d stored at %5u
                                                          \n",i,*(ptr+i),ptr+i);
                }
            printf("Display the array as Pointer\n");
            for (i=0;i<5;i++)
                {
                  printf("The value of a[%d] = %5d stored at %5u\n",i,*(a+i),a+i);
                }
           getch();
          }
```

```
// PTPALYDR.CPP
// THIS PROGRAM DECIDE WETHER GIVEN STRING IS PALYDROM OR NOT WITH THE USE
// OF POINTER AND DISPLAY THE SUITABLE MESSAGE.

#include <stdio.h>
#include <string.h>
#include <conio.h>

void main()
        {
         char *ptr;
         int i=0,n,flag=0;
         clrscr();

         printf("Value of ptr is %u stored at %u\n",ptr,&ptr);
         printf("Please Enter the String:  ");
         gets(ptr);

         n=strlen(ptr);

         printf("Value of ptr is %5u stored at %5u\n",ptr,&ptr);
         printf("The size is %d\n",strlen(ptr));

         while (i < n/2)
                 {
                  if ( *(ptr+i) == *(ptr+n-i-1) )
                          {
                           printf("Address of (ptr+%2d) = %u and content *(ptr+%2d) = %c\n",
                                                                   i,(ptr+i),i,*(ptr+i));
                           flag = 1;
                          }
                  else
                          {
                           flag=0;
                           break;
                          }
                  i++;
                 }

         if (flag==1)
                  printf("Given String is Palyndrome\n");
         else
                  printf("Given String is Not Palyndrome\n");
         getch();
        }
```

```
// POINTE10.CPP ==> ONE DIMENTIONAL ARRAY AND POINTER
// 1. IF POINTER VARIABLE PTR IS INTEGER TYPE THEN PTR+I MEANS 2 BYTES INCEMENT
//    BASE ADDRESS REMAIN CONSTANT

// 2. IF YOU WRITE PTR++ IT MEANS THAT, INCREMENT OF TWO BYTES IN BASE ADDRESS
//    BASE ADDRESS IT SELF INCREASED
//    IF PTR CONTAIN INTEGER   ADDRESS THEN ++ MEANS 2 BYTES
//    IF PTR CONTAIN FLOAT     ADDRESS THEN ++ MEANS 4 BYTES
//    IF PTR CONTAIN CHARACTER ADDRESS THEN ++ MEANS 1 BYTE

// 3. a[i]   MEANS *(a+i) ith INTERGER FROM BASE ADDRESS
//    *(i+a) MEANS *(i+a) WHICH SAME AS a[i]
//    a[i] AND i[a] BOTH ARE SAME

#include <stdio.h>
#include <conio.h>
 void main()
          {
           clrscr();            int a[] = {22,33,44,55,66};   int i;            int *ptr;

           ptr = a;    // ptr = &a[0]
           printf("Base Address of Array a is %u\n",a);
           printf("Base Address of Array a is %u\n",&a[0]);
           printf("Content of Pointer ptr is %u\n",ptr);

           for (i=0;i<=4;i++)
            {
                   printf("a[%d] = %3d stored at %u\t",i,a[i],&a[i]);
                   printf("a[%d] = %3d stored at %u\n",i,*(ptr+i),ptr+i);
            }
           printf("\n\n");

           for (i=0;i<=4;i++)
           {
                   printf("a[%d] = %3d stored at %u\t",i,i[a],&a[i]);
                   printf("a[%d] = %3d stored at %u\n",i,*ptr,ptr);
                   ptr++;
           }
           printf("Content of Pointer ptr is %u\n",ptr);
           for (i=0;i<=4;i++)
           {
                   *(a+i) = a[i] + i[a];  // a[i] = a[i] + a[i]
                   printf("Value of a[%u] = %5d\n",i+a,*(i+a));
            }
           getch();
          }

// 65516   65518  65520  65522  65524
// +----------+---------+---------+---------+------+
// |  22     | 33      | 44      |  55     | 66   |
// +---------+---------+----------+---------+------+
```

```
// POINTER4.CPP
// TWO DIMENTIONAL ADDITION OF ARRAY WITH THE USE OF POINTER.
// EXAMPLE OF ARRAY z[ROW SIZE][COLUMN SIZE]
// *(*(z+i)+j) WHERE 4(COLUMN SIZE) * i PLUS j
// IF i=1 and j=1 then   4 * 1 + 1 = 5 * 2 bytes for int = 10
// Base address of z + 10 = 6

#include <stdio.h>
#include <conio.h>

void main()
        {
         clrscr();
         int z[3][4] = { {1,2,3,4},{5,6,7,8},{9,10,11,12} };
         int a[3][3] = { {1,2,3},{4,5,6},{7,8,9} };
         int b[3][3] = { {1,4,7},{2,5,8},{3,6,9} };
         int c[3][3], i,j;

         printf("Base address of z is %u\n",z);
         printf("Base address of a is %u\n",a);
         printf("Base address of b is %u\n",b);
         printf("Base address of c is %u\n",c);

         for (i=0;i<3;i++)
                 for (j=0;j<4;j++)
                         {
                          printf("z+%d = %u ",i,z+i);
                          printf("conntent of z[%d][%d] is %3d",i,j,*(*(z+i)+j));
                          printf(" & address1 is %u\n",&(z[i][j]));
                         }

         for (i=0;i<3;i++)
                 for (j=0;j<3;j++)
                         *(*(c+i)+j) = *(*(a+i)+j) + *(*(b+i)+j);

         for (i=0;i<3;i++)
                 {
                  for (j=0;j<3;j++)
                          printf("%3d",c[i][j]);
                  printf("\n");
                 }
         getch();
        }
```

```
// POINTER9.CPP ==> USE OF DOUBLE POINTER TO POINT THE VARIABLE

#include <stdio.h>
#include <conio.h>

void main()
   {
     clrscr();
     int i=3;
     int *j;
     int **k;

     j=&i;
     k=&j;

     printf("\n1. Address of i is %u",&i);
     printf("\n2. Address of j is %u",&j);
     printf("\n3. Address of j is %u",&(*k));
     printf("\n4. Address of k is %u\n\n",&k);

     printf("\n1. Content of I is %2d stored at %u",i,&i);
     printf("\n2. Content of I is %2d stored at %u",i,j);
     printf("\n3. Content of I is %2d stored at %u",i,*k);
     printf("\n4. Content of I is %2d stored at %u",**k,*k);
     printf("\n5. Content of I is %2d stored at %u",*j,*k);
     **k = **k + 7;
     printf("\n6. Content of I is %2d stored at %u",*j,*k);
     printf("\n7. Content of I is %2d stored at %u",i,&i);
     getch();
  }

//         i                j                k
//    +----------+     +-----------+    +------------+
//    |          |     |           |    |            |
//    |   3      |     |  65524 |     |  65522   |
//    +----------+     +-----------+    +------------+
//       65524            65522            65520


//    **********    O U T P U T    **********
```

```
// ABCDTRI1.CPP
// IN FIRST LINE G AND G REPEATED TWO TIMES THAT WHY printf("%3c",ch) AT LAST
//
// A  B  C  D  E  F  G  G  F  E  D  C  B  A
// A  B  C  D  E  F<- 06 ->F  E  D  C  B  A
// A  B  C  D  E<---- 12----->E  D  C  B  A
// A  B  C  D<--------18-------->D  C  B  A
// A  B  C<-----------24---------->C  B  A
// A  B<--------------30-------------->B  A
// A<-----------------36----------------->A


#include <stdio.h>
#include <conio.h>
void main()
    {
      clrscr();
      int n,i,j;
      char ch;
      printf("How many rows are there?");
      scanf("%d",&n);

          for (i=n;i>=1;i=i-1)
                {
                  ch = 64;
                  for (j=2*i;j>=1;j = j-1)
                        {
                          if (j==i && j!=n)    // IF N=7 & J=7 STILL ONE SPACE LEFT
                                  printf("%*c",6*(n-j),' ');
                          else if (j>i)
                                  ch = ch + 1;
                          else if (j<i)
                                  ch = ch - 1;
                          printf("%3c",ch);
                        }
                  printf("\n");
                }
          getch();
        }

// A  B  C  D  E  F  G  G  F  E  D  C  B  A
// A  B  C  D  E  F<- 06 ->F  E  D  C  B  A
// A  B  C  D  E<---- 12 ---->E  D  C  B  A
// A  B  C  D<------- 18 ------->D  C  B  A
// A  B  C<---------- 24 ---------->C  B  A
// A  B<------------- 30 ------------->B  A
// A<---------------- 36 ---------------->A
// A  B<------------- 30 ------------->B  A
// A  B  C<---------- 24 ---------->C  B  A
// A  B  C  D<------- 18 ------->D  C  B  A
// A  B  C  D  E<---- 12 ---->E  D  C  B  A
// A  B  C  D  E  F<- 06 ->F  E  D  C  B  A
// A  B  C  D  E  F  G  G  F  E  D  C  B  A
```

```
// A-E.CPP -> PRINT THE CHARACTERS BETWEEN A TO E WITH USE OF ARRAY
// Use WHILE Loop in place of Do__WHILE AND Enter A-A OUTPUT A
// IF you Use DO_WHILE Loop then A-A OUTPUT IS ABC..ZA
// Selection of Loop is also important
#include <stdio.h>
#include <conio.h>
#include <ctype.h>
void main()
  {
        char str[3];
        int i;
        clrscr();
        printf("Enter the character string ");
        gets(str);
        i=toascii(str[0]);
   //   do
        while(toascii(str[2]!=i))
         {
                  printf("%c",i);
                   i++;
                  if(i==91)

                           i=65;
          } // while (toascii(str[2])!=i);
          putchar(str[2]);
getch();
}
```

```
// ASCDES.CPP -> DECIDED WHETHER GIVEN ARRAY IS ORDERED OR NOT
// 1,55,2,88,99,0,44 -> NOT IN ORDER
// 33,66,77,78,89,90 -> IN ORDER
// 1,1,1,1,1,1,1,1,1 -> ALL EMEMENTS ARE EQUAL

#include <stdio.h>
#include <conio.h>

void main()
        {
          int a[50],i,no;     // Declaration of the variables
          char ans = ' ';
          clrscr();

          printf("Enter no of Elements:  ");   // Input the values
          scanf("%d",&no);

          for (i=0;i<no;i++)
                {
                  printf("Enter a[%d] = ",i);
                  scanf("%d",&a[i]);
                }

          for (i=0;i<no-1;i++)            // Logic for Acending, Decending or Not Ordered
                {
                  if (a[i] == a[i+1])
                          continue;
                  else if ( (a[i] - a[i+1]) > 0 && ans != 'A')
                          ans = 'D';
                  else if ( (a[i] - a[i+1]) <= 0 && ans != 'D')
                          ans = 'A';
                  else
                        {
                          ans ='N';
                          break;
                        }
                }

          if (ans == 'A')                           // Displa the message
                  printf("Array sorted in Ascending order\n");
          else if (ans == 'D')
                  printf("Array sorted in Descending Order\n");
          else if (ans == 'N')
                  printf("Not Sorted\n");
          else
                  printf("All elements are equal");
          getch();
        }
```

```
// DATE_NEXT.CPP -> TO PRINT THE NEXT DATE OF THE GIVEN DATE
// 29-02-2004 ==> 01-03-2004    28-02-2004 ==> 29-02-2004
// 31-12-2004 ==> 01-01-2005    31-08-2005 ==> 01-09-2005
//
#include <stdio.h>
#include <conio.h>
void main()
    {
      int d,m,y; int month[] = {31,28,31,30,31,30,31,31,30,31,30,31};
      clrscr();
      printf("Enter the value of Date, month & year : ");
      scanf("%d %d %d",&d,&m,&y);
          if (m==2 && d>=28)
                {
                  if(y%4 == 0 && (y%100!=0 || y%400==0))
                        month[m-1] = 29;  // 1 change with pervious date
                }
          if (d!=month[m-1])
                  d++;
          else if (m!=12)
                {
                  d=1;
                  m++;
                }
          else
                {
                  d=1;
                  m=1;
                  y++;
                }
          printf("The new date is : %d-%d-%d",d,m,y);
          getch();
        }
```

```
// DATEDIF2.CPP -> TO FIND OUT DAY NAME FROM THE GIVEN DATE
// 1-1-1900 To 6-5-1900  = 125   1-1-1900   To 8-5-1902  = 857
// 8-2-2000 To 11-6-2001 = 489   1-2-2000   To 1-2-2000  = 0
// 1-2-2000 To 12-2-2000 = 11    23-10-1996 To 25-4-1997 = 184
// Let Our Initial  Date is : 1-1-1900  Monday
// Check for Second Date is : 1-1-1980  Tuesday
// Check for Third  Date is : 1-1-2000  Saturday
#include <stdio.h>
#include <conio.h>
void main()
    {
      int d1,m1,y1,d2,m2,y2,i,temp;
      long int sum=0;   // CHANGE
      int month[] = {31,28,31,30,31,30,31,31,30,31,30,31};
      char days[][12] = {"Monday","Tuesday","Wedensday",
                     "Thursday","Friday","Saturday", "Sunday"};
      clrscr();
      d1 = 1; m1 = 1; y1 = 1900;
      printf("Enter the Ending -> Day,Month & Year:  ");
      scanf("%d %d %d",&d2,&m2,&y2);
      temp = d1;
          for (i=m1;i<m2 + (y2 - y1) *12;i++)
                {
                  if (i >12)
                        {
                          i = 1;
                          y1++;
                        }
                  if (i==2)
                        {
                          if (y1%4 == 0 && (y1%100!=0 || y1%400==0))
                                  month[i-1] = 29;
                          else
                                  month[i-1] = 28;
                        }
                   sum = sum + (month[i-1] - temp);
                   temp = 0;
                }
          sum = sum + d2 - temp;
          printf("\nTotal Days are :  %ld",sum);
          printf("\nFor Given Date Day is : %s", days[sum%7]);     // CHANGE
          getch();
        }
```

```
// DTOMY.CPP -> TO PRINT THE PREVIOUS DATE OF THE GIVEN DATE
#include <stdio.h>
#include <conio.h>
void main()
        {
          int d,m,y;  int month[] = {31,28,31,30,31,30,31,31,30,31,30,31};
          clrscr();
          printf("enter the value of Date, month & year : ");
          scanf("%d %d %d",&d,&m,&y);
          if (m==3 && d==1)
                {
                  if(y%4 == 0 && (y%100!=0 || y%400==0))
                        month[m-2] = 29;  // 2 is necessary
                }
          if (d > 1)
                 d--;
          else if (m > 1)
                {
                  d=month[m-2];
                  m--;
                }
          else
                {
                  d=31;
                  m=12;
                  y--;
          }
          printf("The new date is : %d-%d-%d",d,m,y);
          getch();
        }
// MAXWOR.CPP -> TO FIND OUT THE MAXIMUM LENGTH WORD FROM THE GIVEN ARRAY
// INBETWEEN TWO WORDS IF THERE ARE MORE THAN ONE SPACE
#include <stdio.h>
#include <conio.h>
void main()
        {
          int j=0,k=0,l=0,m=0;
          char ch[50];
          clrscr();
          printf("Enter the String:  ");      gets(ch);
          do
                {
                  if (ch[m]!=' ' && ch[m]!='\0')
                          j++;
                  else if (j > k)
                        {
                          k = j;
                          j = 0;
                          l = m-k;
                        }
                  else
                          j=0;
                } while (ch[m++]!='\0');
          for (j=l;ch[j]!=' ' && ch[j]!=NULL;j++)
                   putchar(ch[j]);
          getch();
        }
```

```
// PRIL_PR4.CPP    // CALCULATE FACTORIAL  *1   // GENERATE THE SERIES
// 1/1! + 2/2! + 3/3! + 4/4! + ... n TERMS    *2 // 1/1! - 2/2! + 3/3! - 4/4! + ... n TERMS    *3
// 1/0! - 2/1! + 3/2! - 4/3! + ... n TERMS    *4  FIRST TERM ASSUMED SUM = 1.0
#include <stdio.h>
#include <conio.h>
#include <stdlib.h> // atoi() function
void main()
    {
      int n,i,fact;      // *1
      clrscr();
      printf("Enter the value of n to calculate factorial: ");
      scanf("%d",&n);
      fact = 1;
      for (i=1;i<=n;i++)
        {
          fact = fact * i;
        }
      printf("Factorial Value is = %d\n",fact);
//-------------------------------------------------------------------------------
      printf("ASCII + = %5d\n",'+');   // +==> 43 -==> 45
      printf("ASCII - = %5d\n",'-');
      float sum = 0.0;     // 1/1! + 2/2! + 3/3! + 4/4! + ... n TERMS    *2
      fact = 1;
      for (i=1;i<=n;i++)
        {
          fact = fact * i;
          sum  = sum + (float) i/fact;   // 2.00 if n = 4
          printf("%2d/%2d + ",i,fact);
        }
      printf("= %4.2f\n",sum);
//-------------------------------------------------------------------------------
      fact = 1;     // 1/1! - 2/2! + 3/3! - 4/4! + ... n TERMS    *3
      sum  = 0.0;   int sign = 1, value = 45;
      printf("ASCII + = %5d\n",'+');   // +==> 43 -==> 45
      printf("ASCII - = %5d\n",'-');
      for (i=1;i<=n;i=i+1)
        {
          fact = fact * i;
          sum  = sum + (float) (i*sign)/fact;
          sign = sign * -1;
          printf("%2d/%2d %2c ",i,fact,value);
          value = value + (sign*2);
         }
      printf("= %4.2f\n",sum);
//-------------------------------------------------------------------------------
      fact = 1;     // 1/0! - 2/1! + 3/2! - 4/3! + ... n TERMS    *4
      sum  = 1.0;   sign = -1; value = 43;
      printf(" 1/ 1  - ");
      for (i=2;i<=n;i=i+1)
        {
          fact = fact * (i-1);  // BRACKET MUST
          sum  = sum  + (float) (i*sign)/fact;
          sign = sign * -1;
          printf("%2d/%2d %2c ",i,fact,value);
          value = value + (sign*2);
         }
      printf("= %4.2f\n",sum);
      getch();
    }
```

```
// PRIL_PR5.CPP
//   Write a C program to print the following series up to n terms:
//   3, 7, 13, 21, 31, 43 ...
//                                          2
//    Note : (1) k'th term of series is (k+1) - k.
//           (2) Print 10 terms per line.
// n = 181 GIVES UNEXPECTED ANSWER
#include <stdio.h>
#include <conio.h>
#include <math.h>
void main()
    {
      int i,n;
      long int term;    // ONLY INTEGER THEN N = 180 UNEXPECTED ANSWER
      clrscr();
      printf("Enter no of terms:  ");
      scanf("%d",&n);
      printf("First Method...\n");
      for (i=1;i<=n;i=i+1)     // WITHOUT USE OF CONDITION
        {
          term = pow(i+1,2) - i;
          printf("%5ld",term);
        }

//-----------------------------------------------------------------------------
      printf("\nSecond Method...\n");
      for (i=1;i<=n;i=i+1)
        {
          term = pow(i+1,2) - i;
          if (i%10==0)
             printf("%5ld,\n",term);
          else
             printf("%5ld,",term);
        }

// -----------------------------------------------------------------------------
      printf("\nThird Method...\n");
      for (i=1;i<=n;i=i+1)
        {
          term = pow(i+1,2) - i;
          if (i%10==0)
             printf("%5ld,\n",term);
          else if (i==n)
             printf("%5ld.",term);
          else
             printf("%5ld,",term);
        }
      getch();
    }
// -----------------------------------------------------------------------------
```

```
// PRIL_PR6.CPP
// TO GENERATE FIBONACII SERIES ==> 0 1 1 2 3 5 8 13 ...
// TO GENREATE LUCAS SEREIES    ==> 1 3 4 7 11 18 29 ...
// WITH TWO OR THREE VARIABLES
#include <stdio.h>
#include <conio.h>
void main()
        {
          clrscr();
          int n,i,fib,fib1,fib2;
          printf("Enter the no of terms:  ");
          scanf("%d",&n);
          printf("With use of Three Variables\n");
          fib1 = 0;
          fib2 = 1;
          for (i=1;i<=n;i++)
                {
                  fib  = fib1;
                  printf("%5d",fib);
                  fib1 = fib2;
                  fib2 = fib1 + fib;
                }

          printf("\nWith use of Two Variables\n");
          fib1 = 0;
          fib2 = 1;
          for (i=1;i<=n;i++)
                {
                  printf("%5d",fib1);
                  fib2 = fib1 + fib2;
                  fib1 = fib2 - fib1;
                }
          getch();
        }
//---------------------------------------------------------------------------
```

```
// PRIL_PR9.CPP TO FIND AUTOMORPHIC NUMBER FROM THE GIVEN MAXIMUM RANGE
// 1  ==> 1  SQURE IS 1
// 5  ==> 5  SQURE IS 25  AND LAST DIGIT IS ALSO THE NUMBER
// 25 ==> 25 SQURE IS 625 AND LAST TWO DIGITS ARE THE GIVEN NUMBER
// LIST OF AUTOMORPHIC NUMBERS IN BETWEEN 1..10000 ARE GIVEN BELOW
// 1  5  6  25  76  376  625 9376

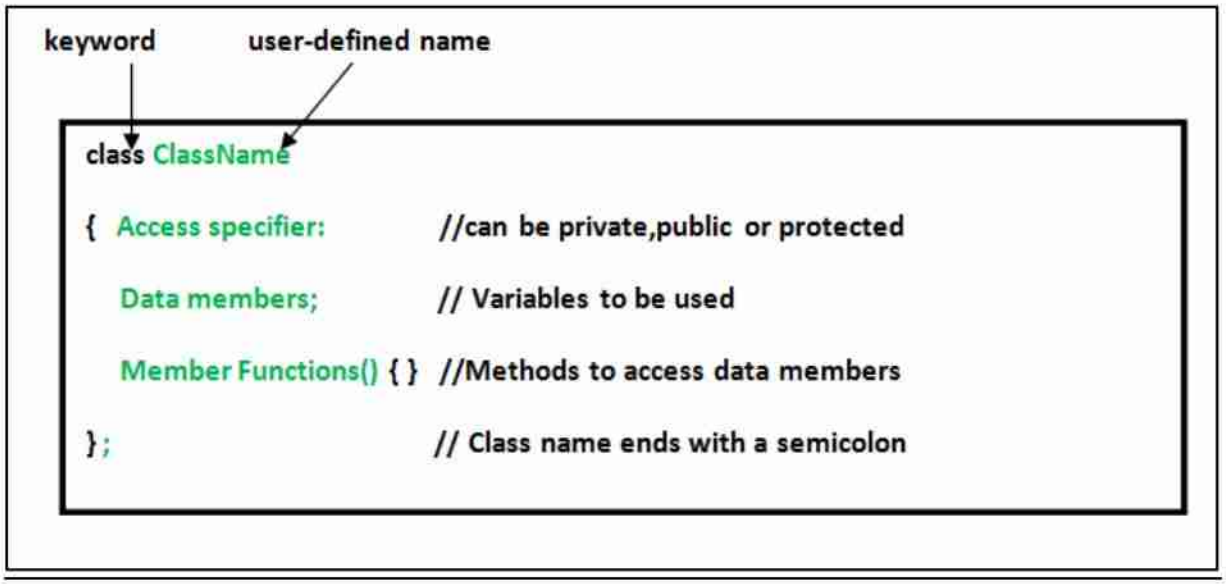
#include <stdio.h>
#include <conio.h>
#include <math.h>
void main()
    {
      clrscr();
      int flag;
      long int i,n;
      long int sum,temp;
      printf("Enter the Maximum No:  ");
          scanf("%ld",&n);
          printf("Following are the Automorphic Numbers\n");
          for (i=1;i<=n;i = i+1)
                {
                  flag = 0;
                  sum  = pow(i,2);
                  temp = i;
                  while (temp >0)
                        {
                          if (temp%10 == sum%10)
                                  flag = 1;
                          else
                                {
                                  flag=0;
                                  break;
                                }
                          temp = temp/10;
                          sum  = sum/10;
                        }
                  if (flag)
                          printf("Squre of %7ld  ==>  %7ld\n",i,i*i);
                }
        getch();
       }
```

```
//TRIAN2.CPP ==> RIGHT JUSTIFIED TRIANGLE WITH TWO FOR LOOPS
//                1
//             1  2
//          1  2  3
//       1  2  3  4
//    1  2  3  4  5

#include <stdio.h>
#include <conio.h>
void main()
        {
          int i,j,space,n;               clrscr();
          printf("Enter The no:  ");     scanf("%d",&n);
          for (i=1;i<=n;i++)
        {
          space = 80 - 3*i;              // BECAUSE OF RIGHT JUSTIFIED
          printf("%*s",space," ");  // AUTOMATICALLY TAKES NEW LINE
          for (j=1;j<=i;j++)
                    printf("%3d",j);
          }
          getch();
        }
```

Concept of Class & Object

- A class in C++ is the building block that leads to Object-Oriented programming.
- A group of objects that share common properties for data part and some program part are collectively called as class.
- It is a user-defined data type, which holds its own data members and member functions, which can be accessed and used by creating an instance of that class.
- A C++ class is like a blueprint for an object.
- Data members are the data variables and member functions are the functions used to manipulate these variables and together these data members and member functions defines the properties and behavior of the objects in a Class.
- A class is defined in C++ using keyword class followed by the name of class. The body of class is defined inside the curly brackets and terminated by a semicolon at the end.



- An **Object** is an instance of a Class. When a class is defined, no memory is allocated but when it is instantiated (i.e. an object is created) memory is allocated.
- Objects are the basic run-time entities in an object-oriented system. They may represent a person, a place, a bank account, a table of data or any item that the program must handle.
- The fundamental idea behind object oriented approach is to combine both data and function into a single unit and these units are called objects.
- Significance of Class and Object in C++

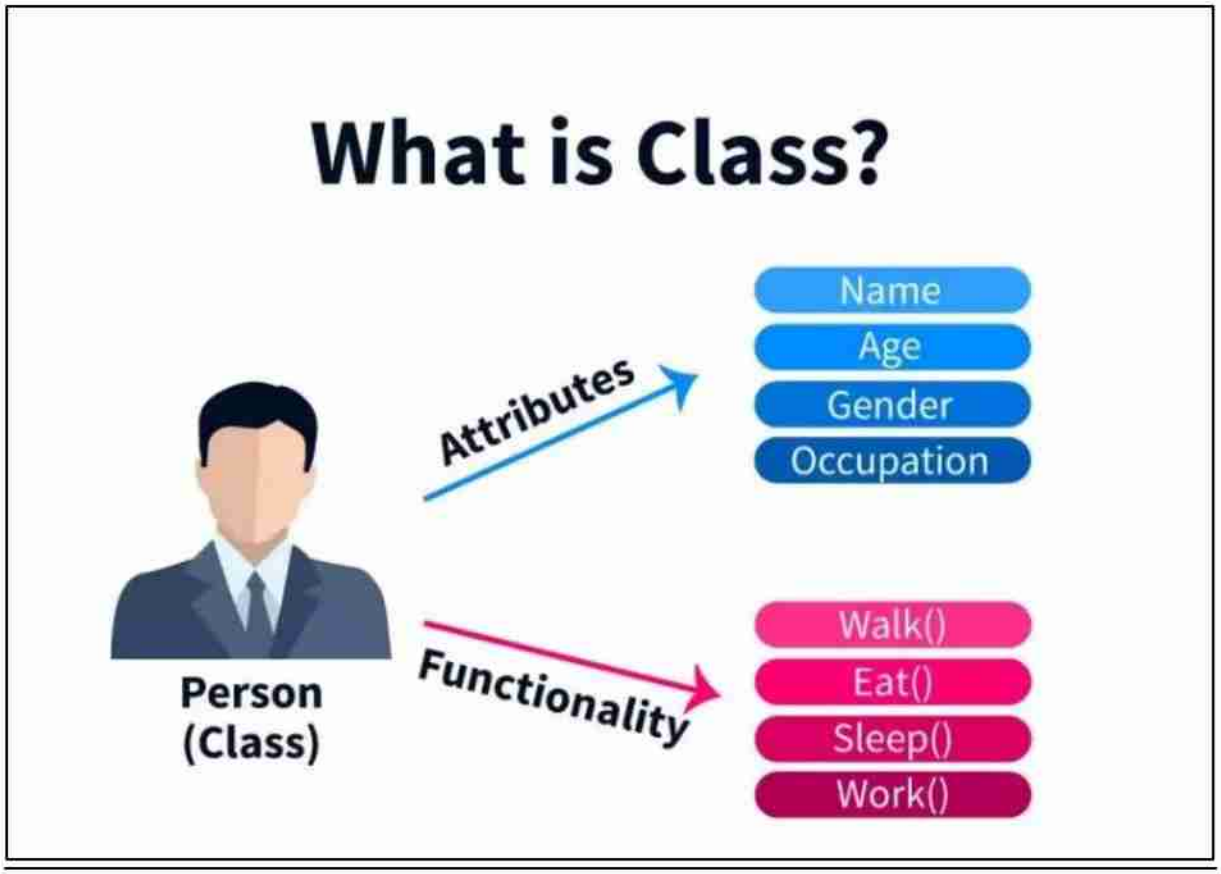
The concept of class and object in C++ makes it possible to incorporate real-life analogy to programming. It gives the data the highest priority using classes. The following features prove the significance of class and object in C++:

- Data hiding or Data Encapsulation: A class prevents the access of the data from the outside world using access specifiers. It can set permissions to restrict the access of the data.

- Code Reusability: You can reduce code redundancy by using reusable code with the help of inheritance. Other classes can inherit similar functionalities and properties, which makes the code clean.

- Data binding: The data elements and their associated functionalities are bound under one hood, providing more security to the data.

- Flexibility: You can use a class in many forms using the concept of polymorphism. This makes a program flexible and increases its extensibility.

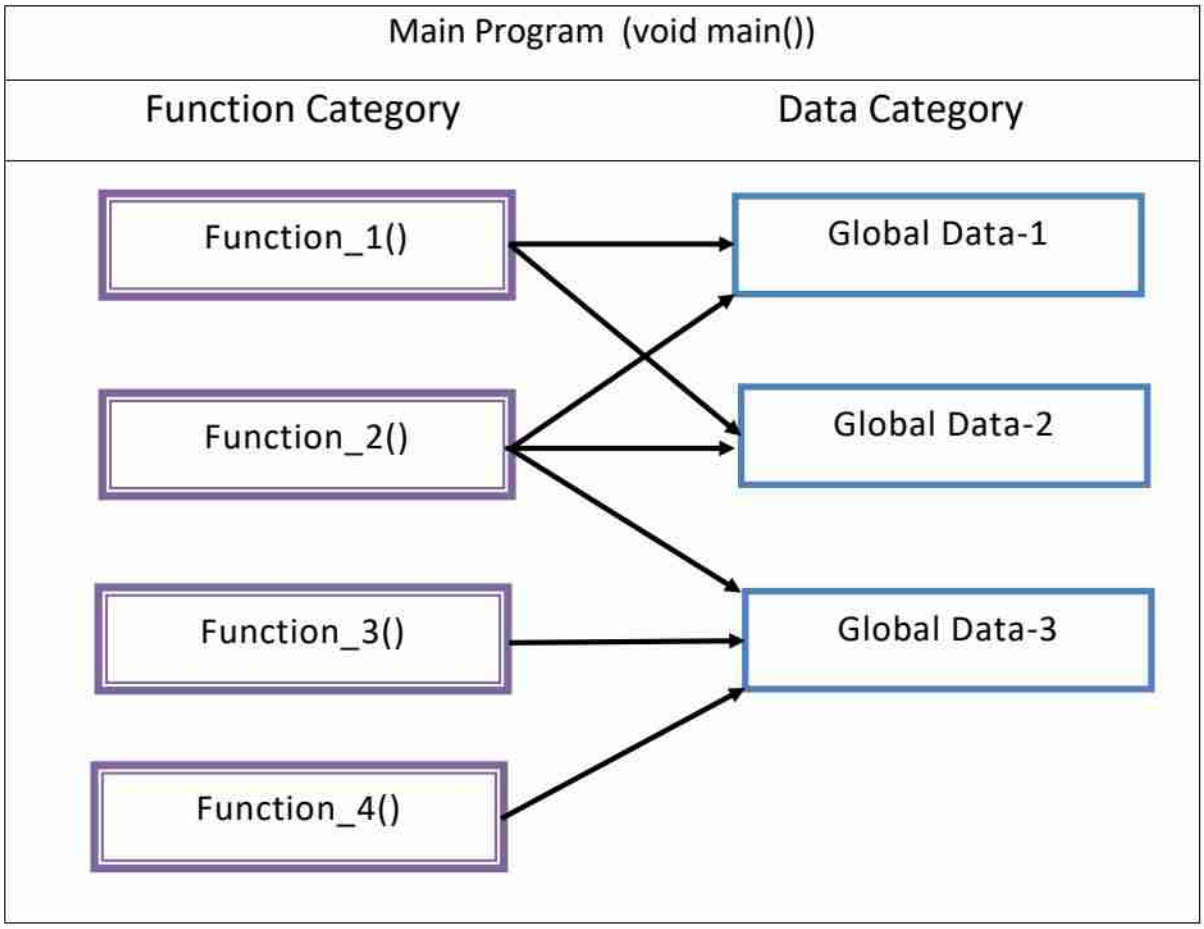


Difference between Class & Object

No	Class	Object
1	In a program, a class is a blueprint for declaring and creating objects.	An object is a class instance that allows programmers to use variables and functions from inside the class.
2	When a class is formed, no memory space is allocated.	When an object is created, it allocates memory space.
3	You can only declare a class once.	A class can be used to create many objects. Relationship of Class➔Object is One-To-Many
4	Class is a logical entity.	An object is a physical entity.
5	We cannot manipulate class as it is not available in memory.	Objects can be manipulated.
6	For example, Mobile phones are the class.	If a mobile phone is the class then iphone, redmi, blackberry, samsung are the objects.

Procedure Oriented Programming Language (POP)

- The problem is viewed as sequence of things to be done such as reading, calculation and printing in the procedure oriented approach.
- Programming in the high-level languages such as COBOL, FORTRAN, C, etc. is known as procedure-oriented programming.
- Procedure-oriented programming basically contains group of instructions known as function. There are multiple functions into the program.
- Procedure oriented programming basically consist of writing a list of instruction or actions for the computer to follow and organizing these instruction into groups known as functions.



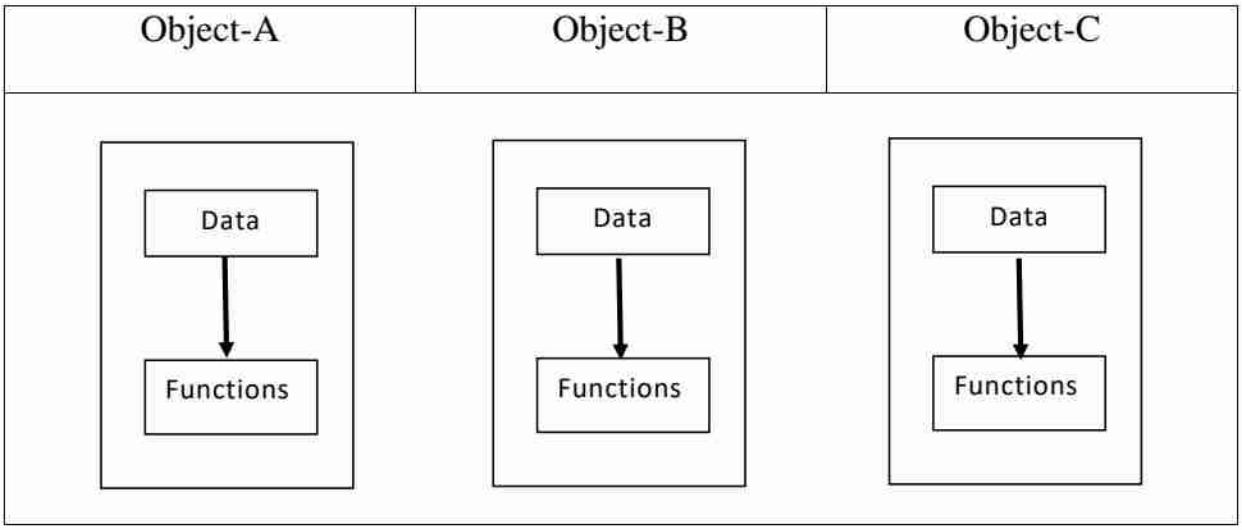
- The disadvantage of the procedure oriented programming languages is:
  - Global data access
  - It does not model real word problem very well
  - No data hiding

- Characteristics of procedure oriented programming:
  - Emphasis is on doing things(algorithm)
  - Large programs are divided into smaller programs known as functions.
  - Functions share global data
  - Data move openly around the system from function to function
  - Function transforms data from one form to another.
  - Employs top-down approach in program design

Object Oriented Programming Language (OOP)

- Object-oriented programming (OOP) is a computer programming model that organizes software design around data, or objects, rather than functions and logic.
- An object can be defined as a data field that has unique attributes and behavior.
- The main aim of OOP is to bind together the data and the functions that operate on them so that no other part of the code can access this data except that function.
- Object-oriented programming or OOP is a type of computer programming in which the objects and their interactions with one another are considered central.
- It is based on the concept that all items in a program such as variables, data structures, and functions should be treated as objects.
- Object-oriented programming aims to increase code reusability by creating different classes for different types of information, which allows developers to easily add features when they need them without having to change existing code.
- OOP focuses on the objects that developers want to manipulate rather than the logic required manipulating them.
- This approach to programming is well-suited for programs that are large, complex and actively updated or maintained.
- Communication is done from one object to another object.

- Relationship between class and object is One-To-Many.



- Features of the Object Oriented programming

  - Emphasis is on doing rather than procedure.
  - Programs are divided into what are known as objects.
  - Data structures are designed such that they characterize the objects.
  - Functions that operate on the data of an object are tied together in the data structure.
  - Data is hidden and can't be accessed by external functions.
  - New data and functions can be easily added.
  - Follows bottom-up approach in program design.

Difference between Structure of C and Class of C++

No	Structure of C	Class of C++
1	One or more data items (Data Members) which are grouped together as a single unit is called Structure	Data members and member functions are grouped together as a single unit is called Class.
2	There is a loose coupling between Data items and functions.	There is a tight coupling between data members and member functions.
3	There is a need to pass structure data items as parameter to function.	There is no need to pass class data members as parameters to member function.
4	Default access specifire of structure data items are public.	Default access specifire of Class Data items are private.
5	Data security is not provided. Data items are accessed by any function	Data security is provided by Data Encapsulation concept. Only member functions can access the Data member of the class

Write a short note on Constructor

1. Constructor is a special member function to create the object and initialized the object.
2. Memory allocation is done when object is created.
3. Object Data members can be initialized only by Constructor
4. There are two types of Constructor:
   a. Default constructor is provided by the Compiler. It creates the object but initialization of the object is not possible. This type of constructor is always without parameters.
   b. User defined constructor is provided by the user. It not only creates the object but also initialized the object. This type of constructor is either parameterized or without parameter.
5. It is a special member function because:
   a. A name of the constructor is same as that of its class name.
   b. It is declared with no return type (Not even void)
   c. User may declare multiple constructors with the use of parameter passing. It is called constructor overloading.
   d. Constructor is called automatically when object of that class is created.
6. In the absence of user defined Constructor, Default constructor is provided by the Compiler at the time of Compilation.

Write a short note on Destructor

1. Destructor is a special member function to destroy the object.
2. Memory is released when object is destroyed.
3. Object Data members can be destroyed only by Destructor
4. There are two types of Destructors:
   a. Default destructor is provided by the Compiler. It destroys the object without any statements. This type of destructor is always without parameters.
   b. User defined destructor is provided by the user. It destroys the object with set of statements. This type of destructor is also without parameters.
5. It is a special member function because:
   a. A name of the destructor is same as that of its class name.
   b. There is a tiled sign (~) is used in front of the destructor name to separate the constructor and destructor.
   c. It is declared with no return type (Not even void)
   d. User may declare only one destructor per class without parameter passing. Destructor overloading is not possible.
   e. Destructor is called automatically when the span of that object is over.
6. In the absence of user defined destructor, Default destructor is provided by the Compiler at the time of Compilation.

Student assignment: Differentiate Constructor Vs. Destructor

Differentiate Constructor Vs. Destructor

No	Constructor	Destructor
1	It is used to create and initialized the object	It is used to destroy the object
2	At the time of creation Memory is occupied	At the time of destroy memory is released.
3	The name of constructor is same as class name.	The name of destructor is same as class name with tiled sign.
4	You may declare multiple user defined constructors with the use of parameters.	You may declare only one user defined destructor with the use of set of statements.
5	Constructor overloading is possible.	Destructor overloading is not possible.
6	There is One-To-Many relationship between Class and Constructor.	There is One-To-One Relationship between Class and destructor.

Write a short note on Copy Constructor

1. Copy Constructor is a special member function.
2. Copy Constructor is automatically called, when reference is made.
3. It is used to copy same class object into another object of same class.
4. It is also used to transfer the object from one function to another function.
5. There are two types of Copy Constructor:
   a. Default Copy Constructor is provided by the Compiler. It is used to copy the object or transfer the object from one function to another function without any message.
   b. User defined Copy Constructor is provided by the user. It is used to copy the object or transfer the object from one function to another function with appropriate message.
6. It is a special member function because :
   a. A name of the Copy Constructor is same as that of its class name.
   b. It is declared with no return type (Not even void)
   c. There is only one Copy Constructor per class.
   d. There is only one parameter pass by reference.
   e. Copy Constructor is called automatically when it is required.
7. The relationship between Class and Copy Constructor is One-To-One.
8. Copy Constructor overloading is not possible.
9. Total Destructors = Constructors + Copy Constructors
10. In the absence of user defined Copy Constructor, Default Copy constructor is provided by the Compiler at the time of Compilation.

Differentiate Constructor Vs. Copy Constructor

No	Constructor	Copy Constructor
1	It is used to create and initialize the object	It is used to copy the same class object into another object and transfer the object from one place to another place.
2	The name of constructor is same as class name.	The name of copy Constructor is same as class name with only one reference parameter.
3	You may declare multiple user defined constructors with the use of parameters.	You may declare only one user defined Copy Constructor with reference parameter.
4	Constructor overloading is possible.	Copy constructor overloading is not possible.
5	There is One-To-Many relationship between Class and Constructor.	There is One-To-One Relationship between Class and Copy Constructor.

1. Difference between Default Constructor Vs. User Defined Constructor
2. Difference between Default Destructor Vs. User Defined Destructor

```
// C_START1.CPP ==> SHOWS ESCAPE SEQUENCE FOR C++
// << CALLED INSERTION OPEATOR OR PUT TO OPERATOR
// >> CALLED EXTRACTION OPERATOR OR GET FORM OPERATOR
// THE IDENTIFIER COUT AND CIN ARE A PREDEFINED OBJECT THAT
REPRESENTS
// THE STANDARD OUTPUT STREAM
// 1. \t  HORIZONTAL TAB
// 2. \n  NEWLINE
// 3. \"  DOUBLE QUOTES
// 4. \'  SINGLE QUOTES
// 5. \b  BACKSPACE
// 6. \r  CARRIAGE RETURN (RETURN TO SAME LINE)
// 7. \a  BEEP NOICE

#include <iostream.h>
#include <conio.h>
int main()
   {
     clrscr();                    // TO CLEAR THE SCREEN
     cout << "MCA Department";
     cout << "\b\b\b\b\b Bhavnagar";
     cout << "\n";
     cout << "\"MCA Department\""<<endl;
     cout << "Bhavanagar\a";
     cout << "\n\'University\'";
     cout << "\rRam\n";
     int a;
     cin >> a;
     cout << a;
     getch();
     return 0;
   }
// O U T P U T
```

```
// C_START2.CPP ==>CALCULATE THE AREA OF THE CIRCLE
// USE OF PRIVATE AND PUBLIC ACCESS SPECIFER
#include <iostream.h>
#include <conio.h>
#include <iomanip.h>
#define PI 3.1415
class area
  {
        private :
                int r;
        public  :
                void getdata()
               {
                cout << "Enter the value of r :  ";
                cin  >> r;
               }
         void cal_display()
               {
                float area;
                area = PI*r*r;
                cout << "Area of Circle is    :  "<<setprecision(4)<<area;
                }
        };
void main()
        {
         clrscr();
         area obj_1,obj_2;  // USER DEFINED OBJECT CREATION
         obj_1.getdata();
         obj_1.cal_display();
         obj_2.getdata();
         obj_2.cal_display();
         getch();
        }
```

```
// PROG2.CPP ==> DECLARE CLASS WITH DATA MEMBERS & MEMBER
FUNCTIONS
#include <iostream.h>
#include <iomanip.h>
#include <conio.h>
class student
      {  private:    int no;
                     char name[10];
                     int mark1,mark2,total;
                     float per;
         public:
                     void getdata();
                     void caldata();
                     void disdata();
      };
void student::getdata()
      {
         clrscr();
         cout<<"Enter the no: ";          cin >> no;
         cout<<"Enter the name: ";      cin >> name;
         cout<<"Enter the mark1: ";     cin >> mark1;
         cout<<"Enter the mark2: ";     cin >> mark2;
        }
void student::caldata()
      {
         total=mark1 + mark2;
         per = total/2.0;
      }
void student::disdata()
      {
         cout<<"NO"<<setw(15)<<"NAME"<<setw(14)<<"TOTAL";
         cout<<<<setw(5)<<"%"<<endl;

         cout<<no<<setw(13)<<name<<setw(13)<<total
         cout<<setw(10)<<setprecision(2)<<per<<endl;
        }
```

```
void main()
    {
       student s1,s2;
       s1.getdata();
       s1.caldata();
       s1.disdata();
       s2.getdata();
       s2.caldata();
       s2.disdata();
       getch();
    }
```

Example -1: A class with Constructors and Destructor

- There are 2 data members and size is 4+4 = 08 bytes
- There are 5 member functions
- Objects are created in the order f1➔f2➔f3 with the use of Constructor member functions
- Objects are destroyed in the reverse order of Creation (f3➔f2➔f1) with the use of destructor member function. Destructor member function is called automatically when main() scope is over.

```
#include <conio.h>
#include <iostream.h>
#include <iomanip.h>
class Fraction
  {
   private : float num;      // Numerator
             float denom;    // Denominator
   public  :
            Fraction ();       // Constructor member function Without Parameter
            Fraction (float);  // Constructor member function with one Parameter
            Fraction (float ,float ); // Constructor member function with Two Parameters
           ~Fraction()  // Destructor Member function
            {cout<< "Fraction Destructor Called..."<<endl;}
            void div_display(); // Normal Member function
  };

Fraction::Fraction()
  {
   cout << "No Parameter   Constructor Called..."<<endl;
   num = 0.0;  denom=1.0;
  }
```

```
Fraction::Fraction(float n)
  {
   cout << "One Parameter  Constructor Called..."<<endl;
   num = n;  denom=1;
  }

Fraction::Fraction(float n,float d)
  {
   cout << "Two Parameters Constructor Called..."<<endl;
   num = n;  denom = d;
  }

void Fraction::div_display()
  {
   cout << "Numerator   value is : "<<num<<endl;
   cout << "Denominator value is : "<<denom<<endl;
   cout << "Division Answer is   : "<< num/denom<<endl<<endl;
  }

void main()
  {
   clrscr();
   Fraction f1,f2(4.0),f3(5.0,3.0);
   cout <<endl;
   f1.div_display();
   f2.div_display();
   f3.div_display();
   getch();
  }
```

Example -2: A class with Constructors and Destructor

- There are 3 data members and size is 2+8+1 = 11 bytes
- There are 4 member functions
- It is a example of constructor overloading by passing parameters.
- Constructor parameter initialization is possible from **Right-To-Left only**
- In-between Constructor Parameter initialization is not Possible
- All Constructor parameters initialization are not possible. At least one should be dynamic
- Objects are created in the order e➔c➔d➔f with the use of Constructor member functions
- Objects are destroyed in the reverse order of Creation (f➔d➔c➔e) with the use of destructor member function. Destructor member function is called automatically when main() scope is over.

```
#include <conio.h>
#include <iostream.h>
#include <iomanip.h>
class Value
 {      private:
                int a;    double b;    char ch;
        public:
                Value();
                Value(int ,double, char );
                void display();
                ~Value();
  };
Value::Value(int m, double n=300.01, char c='X')
  {
        cout <<"Constructor is called with parameters"<<endl;
        a = m;          b = n;          ch = c;
  }
```

```
Value::Value()
 {
        cout <<"Constructor is called without parameters"<<endl;
        a    = 0;       b    = 0.0;    ch = 'Z';
  }

void Value::display()
 {
        cout.precision(2);
        cout << "Integer Value is  : "<<a<<endl;
        cout << "Double Value is   : "<<b<<endl;
        cout << "Character Value is: "<<ch<<endl;
  }

Value::~Value()
  {
    cout <<"\Destructors Called..."<<endl;
  }

void main()
 {
        clrscr();
        char str= 'T';
        Value e;                        e.display();
        Value c(10,100.80,str);         c.display();
        Value d(20,200.67,'B');         d.display();
        Value f(30);                    f.display();
        getch();
 }
```

Example -3: A class with Constructors and Destructor

- There are 1 data members and size is 2 bytes
- There are 3 member functions
- It is a example of Array of Objects
- Objects are created in the order obj[0]➔obj[1]➔obj[2] with the use of Constructor member functions
- Objects are destroyed in the reverse order of Creation (obj[2]➔obj[1]➔obj[0]) with the use of destructor member function. Destructor member function is called automatically when main() scope is over.
-

```
#include <conio.h>
#include <iostream.h>
#include <iomanip.h>

class Sample
  {
    private :int i;              // Data Members
    public :                     // Member functions
          Sample();              // Constructor Member function
          void display();        // Normal Member function
          ~Sample();             // Destructor Member function
  };

Sample::Sample()
  {
     cout << "Constructor is called without Paramters"<<endl;
     cout << "Enter the Value of i :";
     cin >> i;
  }
```

```
void Sample::display()
   {
        cout << "The value of i is : "<<i<<endl;
   }

Sample::~Sample()

   {
        cout << "Destructor is Called..."<<i<<endl;
   }

void main()
   {
     clrscr();
     Sample obj[3];
     for (int i=0;i<3;i++)
        {
          obj[i].display();
        }
     getch();
   }
```

Example -4: A class with Constructors, Copy constructor and Destructor

- There are 2 data members and size is 2+20 = 22 bytes
- There are 4 member functions.
- It is an example of Copy constructor.
- It is used to copy same class object into another object of same class.
- Total Number of Destructors = No. Of Constructors + No. Of Copy Constructors

```
#include <conio.h>
#include <iostream.h>
#include <iomanip.h>
#include <string.h>
class Item
 {
        private:
                int item_no;
                char item_name[20];
        public:
                Item() {cout<< "Constructor without Parameters"<<endl;}
                Item(int a,char name[20])
                        {
                         cout<<"Constructor with two Parameters"<<endl;
                         item_no=a;
                         strcpy(item_name,name);
                        }
                Item(Item &x)
                       {
                         cout<<"Copy Constructor"<<endl;
                         item_no   = x.item_no;
                         strcpy(item_name,x.item_name);
                       }
```

```
            void display()
                {
                  cout << item_no<<setw(10)<<item_name<<endl;
                }
            ~Item() { cout << "Destructor Called..."<<endl;}
 };
void main()
 {
       clrscr();
       Item a(100,"Ram");
       cout << " Object a==> ";   a.display();
       Item b(a);
       cout << " Object b==> ";   b.display();
       Item c = a;
       cout << " Object c==> ";   c.display();
       getch();
        }
// Output
Contractor with two parameters
Object a==> 100      Ram
Copy Constructor
Object b==> 100      Ram
Copy Constructor
Object c==> 100      Ram
Destructor Called...
Destructor Called...
Destructor Called...
```

Example -5: A class with Constructors, Copy constructor and Destructor

- There are 2 data members and size is 2+20 = 22 bytes
- There are 5 member functions.
- It is an example of Copy constructor.
- It is used to copy same class object into another object of same class.
- Total Number of Destructors = No. Of Constructors + No. Of Copy Constructors

```
#include <conio.h>
#include <iostream.h>
#include <iomanip.h>
#include <string.h>
class Value
 {
        private:
                int i;
                char name[20];
        public:
                Value();
                Value(Value &z);
                Value(int k,char str[]);
                ~Value();
  };

Value::Value(int k,char str[])
     {
        cout << "Constructor with Two parameters"<<endl;
        i=k;
        strcpy(name,str);
       }
```

```
Value::Value(Value &z)
     {
        cout << "Copy Constructor Called"<<endl;
        i=z.i;
        strcpy(name,z.name);
     }

Value::Value()
       {
         cout << "Constructor Without Parameter"<<endl;
         i=100;
         strcpy(name,"Bhavnagar");;
       }

Value::~Value()
       {
         cout << "\tDestructor is Call"<<endl;
         cout << "\tThe value of i    : "<<i<<endl;
         cout << "\tThe value of name : "<<name<<endl;
         cout << "\tRelease the memory of object"<<endl;
         cout <<endl;
        }
void main()
 {
       clrscr();
       Value v1;           // CONSTRUCTOR
       Value v2(v1);       // COPY CONSTRUCTOR
       Value v3(200,"MCA BCA");  // CALL BY VALUE
       cout << "Destructor call start In Reverse order..."<<endl;
  }
```

Assignment-1

Q.1 Write a C++ program to find out maximum number from the given Array

```
#include <iostream.h>
#include <conio.h>
#include <iomanip.h>
class Sort
        {
         private:        int value[100];
         public :        Sort(int );
                         int  maxvalue(int );
                         ~Sort() { cout << " Destructor Called..."<<endl;}
        };
Sort:: Sort(int n)
        {
        cout << "Constructor Called..."<<endl;
        for (int i=0;i<n;i++)
         {      cout <<"Enter ["<<i<<"]=" ;
                cin  >> value[i];
         }
        }
int Sort::maxvalue(int n)
        {
        int max=value[0];
        for(int i=1;i<n;i++)
         {
                 if (max < value[i])
                         max = value[i];
         }
        return max;
        }
void main()
        {
        int n;
        clrscr();
        cout<< "How many values are there?  : ";          cin>>n;
        Sort s(n);
        cout <<"Max value is :" << s.maxvalue(n);
        getch();
   }
// O U T P U T
```

Q.2 Write a C++ program to find out minimum number from the given Array

- Structure of C++ program is given below:
- Class Minimum

```
{
        Private: float value [100];
        Public :  Minimum()
                  int minimumvalue (int n);
                  ~Minimum();
};
```

- You are instructed to write appropriate function code for the above class.
- Write appropriate main()

Q.3 Write a C++ program to find out whether given string is palindrome or not

- Structure of C++ program is given below:
- Class Palindrome

```
{
        Private :    char str[50];
                     int flag;
        public  :    Paylindrome();
                     void find_palindrome();
                     void display();
                     ~Palindrome();
};
```

- Get the string with the use of Palindrome() constructor member function
- Write the logic in find_palindrome() normal member function and set flag=0 if given string is not palindrome or flag=1 if given string is palindrome
- Based on flag value display appropriate message with the use of display()
- Write the code for ~Palindrome() destructor member function
- Write appropriate main()

- Input: madam ➔ Palindrome
- Malayalam ➔ Palindrome

Q.4 Write a C++ program to sort given array

- Structure of C++ program is given below:
- Class Sort_value

```
{
        Private :    int  a[50];
        public  :    Sort_value (int n);
                     void sort (int n);
                     void display(int n);
                     ~Sort_value();
};
```

- Get the array value based on parameter n with the use of constructor member function.
- Write the logic in Sort(int n) member function
- Display sorted value with the use of display (n)
- Write the code for ~Sort_value()
- Write appropriate main()

Q.5 Write a C++ program to find out given array is sorted or not

- Structure of C++ program is given below:
- Class Array_5

```
{
        Private :    int  a[50];
                     char answer;
        public  :    Array_5 (int n);
                     void sort_decide (int n);
                     void display();
                     ~Array_5();
};
```

- Get the array value based on parameter n with the use of constructor member function.
- Write a logic in sort_decide(int n) and set answer value as followes

Answer =A ➔ Array is in Ascending Order

Answer = B ➔ Array is in Descending order

Answer = C ➔ Array is in no order

Answer = D ➔ All elements are equal

- Based on above answer appropriate message will be display by display() member function.
- Write the code for ~Array_5()
- Write appropriate main()
- Example

9	12	14	22	26	39	45	48
a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]

Answer =A ➔ Array is in Ascending Order

67	55	31	26	24	20	15	14
a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]

Answer = B ➔ Array is in Descending order

25	36	56	66	32	77	88	99
a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]

Answer = C ➔ Array is in no order

55	55	55	55	55	55	55	55
a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]

Answer = D ➔ All elements are equal

```
// This Program demonstrate the Call By Value and Call By Address through the
function
#include <stdio.h>

// Function for Call by Value
void callByValue(int a, int b)
{
   int temp = a;
   a = b;
   b = temp;
   printf("\nInside callByValue function:");
   printf("\nA = %d, B = %d", a, b);
}
// Function for Call by Address (Reference)
void callByAddress(int *a, int *b)
{
   int temp = *a;
   *a = *b;
   *b = temp;
   printf("\nInside callByAddress function:");
   printf("\nA = %d, B = %d", *a, *b);
}
int main()
{
   int x = 5, y = 10;

   printf("Initial values:\nX = %d, Y = %d\n", x, y);
   // Call by Value Example
```

```
    printf("\n--- Call By Value ---");
    callByValue(x, y);
    printf("\nAfter callByValue (in main):");
    printf("\nX = %d, Y = %d\n", x, y);
    // Call by Address Example
    printf("\n--- Call By Address ---");
    callByAddress(&x, &y);
    printf("\nAfter callByAddress (in main):");
    printf("\nX = %d, Y = %d\n", x, y);
    return 0;
}
// Program Output is as Follows
Initial values:
X = 5, Y = 10

--- Call By Value ---
Inside callByValue function:
A = 10, B = 5
After callByValue (in main):
X = 5, Y = 10

--- Call By Address ---
Inside callByAddress function:
A = 10, B = 5
After callByAddress (in main):
X = 10, Y = 5
```

Difference between Call By Value & Call By Reference

No	Call By Value	Call By Reference
1	While calling a function, we pass the values of variables to it. Such functions are known as "Call By Values".	While calling a function, instead of passing the values of variables, we pass the address of variables(location of variables) to the function known as "Call By References.
2	In this method, the value of each variable in the calling function is copied into corresponding dummy variables of the called function.	In this method, the address of actual variables in the calling function is copied into the dummy variables of the called function.
3	With this method, the changes made to the dummy variables in the called function have no effect on the values of actual variables in the calling function.	With this method, using addresses we would have access to the actual variables and hence we would be able to manipulate them.
4	In call-by-values, we cannot alter the values of actual variables through function calls.	In call by reference, we can alter the values of variables through function calls.
5	This method is preferred when we have to pass some small values that should not change.	This method is preferred when we have to pass a large amount of data to the function.

Example-1 This program demonstrate call by Value with the use of Copy Constructor

```
#include <conio.h>
#include <iostream.h>
#include <iomanip.h>
class Money
{
        private :       int rupees,paisa;
         public  :      Money();
                        Money(int , int);
                        Money(Money &);
                        void sum(Money,Money);
                        void putdata();
                        ~Money(){cout << "Destructor Calledd..."<<endl;}
};
Money :: Money()
{
        cout << "Consturctor Called without Parameter"<<endl;
        rupees = 0; paisa = 0;
}
Money :: Money(int r,int p)
{
         cout << "Constructor Called with paramters..."<<endl;
         rupees = r;   paisa  = p;
}
```

```
Money::Money ( Money &z)
{
        cout << "Copy Constructor Called..."<<endl;
        rupees = z.rupees;  paisa  = z.paisa;
}
void Money :: sum(Money z1, Money z2)
{
         paisa  = z1.paisa + z2.paisa;
         rupees = paisa/100;
         paisa  = paisa%100;
         rupees = rupees + z1.rupees + z2.rupees;
}
void Money :: putdata()
{
        cout.precision(2); cout.width(10);// Rupees width set but paisa width not set
        cout<< rupees<<" Rupees and "<<paisa<<" Paisa"<<endl;
 }
void main()
{
      clrscr();    Money m1(5,84),m2(8,38),m3;
      cout << "M1 Amount is ";   m1.putdata();
      cout << "M2 Amount is ";   m2.putdata();
      m3.sum(m1,m2);
      cout << "M3 Amount is ";   m3.putdata();
      getch();
}
```

⇨ How many Constructors, Copy Constructors and Destructors called?

Example-2 This program demonstrate call by Reference **(No Need of Copy Constructor)**

```
#include <conio.h>
#include <iostream.h>
#include <iomanip.h>
class Money
 {
          private :     int rupees,paisa;
          public  :     Money(int , int);
                        Money();
                        void sum(Money & , Money &);
                        void putdata();
                        ~Money() { cout << "Destructor Called..."<<endl;}
}
Money :: Money(int r,int p)
{
         cout << "Constructor With Parameter Called..."<<endl;
         rupees = r;     paisa  = p;
}
Money::Money()
{
        cout << "Constructor Without Parameter Called..."<<endl;
        rupees = 0;              paisa = 0;
}
```

```
void Money :: sum(Money &m1, Money &m2)
{
        paisa  = m1.paisa + m2.paisa;
        rupees = paisa/100;
        paisa  = paisa%100;
        rupees = rupees + m1.rupees + m2.rupees;
}
void Money :: putdata()
{
        cout.precision(2);
        cout.width(3);    // Rupees width set but paisa width not set
        cout<< rupees<<" Rupees and "<<paisa<<" Paisa"<<endl;
}
void main()
{
        clrscr();
        Money m1(5,84),m2(8,38),m3;
        m3.sum(m1,m2);
        cout << "M1 Amount is ";   m1.putdata();
        cout << "M2 Amount is ";   m2.putdata();
        cout << "M3 Amount is ";   m3.putdata();
        getch();
}
```

⇨ How many Constructors, Copy Constructors and Destructors called?

Concept of Static Data Members

- In **C++**, static is a keyword that can be used with **data members** (variables) and **member functions** of a class.
- It allows sharing of a variable or function **among all objects of the class** rather than having a separate copy for each object.
- A **static data member** belongs to the **class**, not to any specific object.
- Only **one copy** of the static variable is shared among all objects.
- Static data members of the class known as "Class Variables".
- Static member of the class does not contribute to the size of the object- Occupying space in the memory.
- It is **declared inside the class** using the static keyword but **initialized outside** the class using scope resolution:: operator.
- Commonly used for **counters, shared configurations, and efficient memory usage**.

Concept of Static Member Functions

- A **static member function** in C++ is a function that belongs to the class rather than any specific object of the class. It is declared using the static keyword inside the class definition.
- It can be called without creating an object. You may call it by <<class_name>>::<<Function_name()>>
- Only has access to **static data members** or other **static functions**.
- Useful when a function's logic is independent of object state.

Example -1: Static Data Members

```
#include <iostream.h>
#include <iomanip.h>
#include <conio.h>
class Date
{private :    int dd,mm,yyyy;
             static char month[][4];
public :     Date(int, int, int);        // CONSTRUCTOR0
             void disp_dt();             // MEMBER FUNCTION
             void disp_dt1();            // MEMBER FUNCTION
};
// DEFINATION OUT SIDE THE CLASS DECLARATION - IMPORTANT
char Date:: month[][04] = { "Jan","Feb","Mar","Apr","May",
"Jun","Jul","Aug","Sep","Oct","Nov","Dec"};
Date::Date(int d,int m, int y)
      {
            dd = d; mm = m; yyyy = y;
      }
void Date::disp_dt()
      {
            cout << "Date is Display in the format of (YYYY-MM-DD) ==> ";
            cout << yyyy<<"-"<<mm<<"-"<<dd<<endl;
      }
void Date::disp_dt1()
      {
            cout << "Date is Display in the format of (DD-Mon-YYYY)==> ";
            cout << dd <<"-"<<month[mm-1]<<"-"<<yyyy<<endl;
      }
void main()
      {
            clrscr();
            Date D1(5,11,2000), D2(22,2,1999);
            cout << "Size of Object D1: "<<sizeof(D1)<<endl;
            cout << "Size of Object D2: "<<sizeof(D2)<<endl;
            D1.disp_dt();
            D1.disp_dt1();
            D2.disp_dt();
            D2.disp_dt1();
            getch();
      }
```

<table>
<tr><th>Object- D1 (6 Bytes)</th><th>Object-D2 (6 Bytes)</th><th>Description</th></tr>
<tr><td>
<table>
<tr><td>dd</td><td>mm</td><td>yyyy</td></tr>
<tr><td>5</td><td>11</td><td>2000</td></tr>
<tr><td>2 Bytes</td><td>2 Bytes</td><td>2 Bytes</td></tr>
</table>
</td><td>
<table>
<tr><td>dd</td><td>mm</td><td>yyyy</td></tr>
<tr><td>22</td><td>02</td><td>1999</td></tr>
<tr><td>2 Bytes</td><td>2 Bytes</td><td>2 Bytes</td></tr>
</table>
</td><td><u>Object Memory</u>
Data Members
06 + 06 = 12 bytes</td></tr>
<tr><td colspan="2">char Date:: month[][04] = { "Jan","Feb","Mar","Apr","May", "Jun","Jul","Aug","Sep","Oct","Nov","Dec"};

<table>
<tr><td>Row No</td><td>Column[0]</td><td>Column[1]</td><td>Column[2]</td><td>Column[3]</td></tr>
<tr><td>Month[0]</td><td>J</td><td>a</td><td>n</td><td>\0</td></tr>
<tr><td>Month[1]</td><td>F</td><td>e</td><td>b</td><td>\0</td></tr>
<tr><td>Month[2]</td><td>M</td><td>a</td><td>r</td><td>\0</td></tr>
<tr><td>Month[3]</td><td>A</td><td>p</td><td>r</td><td>\0</td></tr>
<tr><td>Month[4]</td><td>M</td><td>a</td><td>y</td><td>\0</td></tr>
<tr><td>Month[5]</td><td>J</td><td>u</td><td>n</td><td>\0</td></tr>
<tr><td>Month[6]</td><td>J</td><td>u</td><td>l</td><td>\0</td></tr>
<tr><td>Month[7]</td><td>A</td><td>u</td><td>g</td><td>\0</td></tr>
<tr><td>Month[8]</td><td>S</td><td>e</td><td>p</td><td>\0</td></tr>
<tr><td>Month[9]</td><td>O</td><td>c</td><td>t</td><td>\0</td></tr>
<tr><td>Month[10]</td><td>N</td><td>o</td><td>v</td><td>\0</td></tr>
<tr><td>Month[11]</td><td>D</td><td>e</td><td>c</td><td>\0</td></tr>
</table>
</td><td><u>Class Memory</u>

Static Data Members

12 months * 4 = 48 bytes</td></tr>
<tr><td colspan="2">1. Date(int,int,int)
2. Void disp_dt()
3. Void disp_dt1()</td><td><u>Member Functions</u></td></tr>
</table>

Example-2 Static Data Members and Static Member Functions

- All the object of the class share only one copy of the member.
- Static data members of the class known as "Class Variables".
- Static member of the class does not contribute to the size of the object- Occupying space in the memory.
- It is used for recording data common to all objects of a class.
- The definition of a static member is equivalent to an external variable.
- One must define the static member outside class declaration.
- A static member function can access the names of static members

```
#include <stdio.h>
#include <conio.h>
#include <iomanip.h>

class Sample

{
private:
        static int count;
public :
        Sample();
        static void display();
        ~Sample();
};
int sample::count=0; // Definition outside the class
Sample::Sample()
    {
        cout << "Constructor Called...<<++count<<endl;
    }
Void Sample::display()
    {
        cout<< "Counter Value = "<<count<<endl;
    }
Sample::~Sample()
    {
        cout<<"Destructor Counter value: "<<count--<<endl;
    }
```

```
void main()
{
        clrscr();
        cout << "Before the object Created"<<endl;
        Sample::display();

        Sample obj1[3],obj2,obj3,obj4;
        for (int i=0;i<3;i++)
            {
               Cout<<"Size of Obj1["<<i<<"] =" sizeof(obj1[i])<<endl;
            }

        cout<< "Size of obj1 is :"<<sizeof(obj1)<<endl; // 1 + 1 + 1 = 03
        cout<< "Size of obj2 is :"<<sizeof(obj2)<<endl; //Default size is 1
        cout<< "Size of obj3 is :"<<sizeof(obj3)<<endl; //Default size is 1
        cout<< "Size of obj4 is :"<<sizeof(obj4)<<endl; //Default size is 1

        cout << "After the object Created"<<endl;
        Sample::display();
        cout << "Destructor function call..."<<endl;
        getch();
}
```

<table>
<tr><td>Obj1[0]</td><td>Obj1[1]</td><td>Obj1[2]</td><td>Obj2</td><td>Obj3</td><td>Obj4</td><td>Description</td></tr>
<tr><td>Default
1 byte</td><td>Default
1 byte</td><td>Default
1 byte</td><td>Default
1 byte</td><td>Default
1 byte</td><td>Default
1 byte</td><td><u>Object Memory</u>
Data Members
01 + 01 + 01 + 01 +01 + 01
= 06 Bytes</td></tr>
<tr><td colspan="6">Static int Count = 0</td><td><u>Class Memory</u>

Static Data Members

1 * 2 = 2 bytes</td></tr>
<tr><td colspan="6">1. static void display();</td><td><u>Static Member Functions</u></td></tr>
<tr><td colspan="6">1. Sample()
2. ~Sample()</td><td><u>Non Static Member Functions</u></td></tr>
</table>

Example-3 Static Data Members and Static Member Functions

```
#include <conio.h>
#include <iostream.h>
#include <stdio.h>
#include <string.h>
class Vowels
{
private :     static int counter[5]; static char v[5];
             char str[50];

public :      Vowels(); { cout << "Enter the string: "; gets(str);}
             void cal_vowels();
             static void find(char);
             static void display();
             ~Vowels() { cout <<Destructor Called…"<<endl;}
};

int vowels::counter[] = {0};
char vowels::v[5] = { 'a','e','i','o','u'};

void Vowels:: find(char ch)
      {
            for (int i=0;i<5;i++)
                {
                  if (ch==v[i])
                        {
                              counter[i]++;
                              break;
                        }
                }
      }
void Vowels::cal_vowels()
      {
            int i=0;
            while (str[i]!=NULL)
                  {
                        vowels::find(str[i]);
                        i++;
                  }
      }
```

```
void Vowels::display()
	{
		int i;
		for (i=0;i<5;i++)
		{
			cout <<v[i]<<"="<<counter[i]<<endl;
		}
	}
void main()
	{
		clrscr();
		Vowels ::display();

		Vowels object_1;
		Vowels object_2;

		object_1.cal_vowels();
		Vowels::display();

		object_2.cal_vowels()
		cout << "Total Number of Vowels are : "<<endl;
		Vowels :: display();
		 getch();
	}
```

<table>
<tr><th>Object_1 (50 Bytes)</th><th>Object_2 (50 Bytes)</th><th>Description</th></tr>
<tr><td>str
<table><tr><td>t</td><td>h</td><td>e</td><td>r</td><td>e</td><td></td><td>i</td><td>s</td><td></td><td>a</td><td></td><td>p</td><td>e</td><td>n</td><td>\0</td></tr></table></td><td>Str
<table><tr><td>e</td><td>l</td><td>e</td><td>p</td><td>h</td><td>a</td><td>n</td><td>t</td><td>\0</td></tr></table></td><td><u>Object Memory</u>
Data Members
50 + 50 = 100 bytes</td></tr>
<tr><td colspan="2">Int Counter[5]
<table><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr></table>Char v[5]
<table><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td></tr><tr><td>a</td><td>e</td><td>i</td><td>o</td><td>u</td></tr></table></td><td><u>Class Memory</u>

Static Data Members

5*2 + 5*1 = 15 bytes</td></tr>
<tr><td colspan="2">1. static void find(char);
2. static void display();</td><td><u>Static Member Functions</u></td></tr>
<tr><td colspan="2">1. Vowels()
2. void cal_vowels();
3. ~Vowels()</td><td><u>Non Static Member Functions</u></td></tr>
</table>

Example-4 Static Data Members and Static Member Functions

```
#include <conio.h>
#include <iostream.h>
#include <iomanip.h>
class Sales
{
        private :     char month[10];     // NAME OF THE MONTH
                      int sal_u;              // SALES IN THE UNIT FOR GIVEN MONTH
                      float sal_r,sal_a; // SALES RATE AND FOR GIVEN MONTH
                      static int unit_tot;
                      static float amt_tot;
        public :
                      void getdata();
                      void find_tot();
                      static void display();
};

int Sales:: unit_tot = 0;
float Sales:: amt_tot = 0.0;

void Sales::getdata()
        {
                cout << "Month Name : "; cin >> month;
                cout << "Number of Units Sales : "; cin >> sal_u;
                cout << "Sales rate per Unit : "; cin >> sal_r;
        }

void Sales::find_tot()
        {
                cout <<setw(15)<<month<<setw(7)<<sal_u<<setw(7)<<sal_r;
                sal_a = sal_u * sal_r;
                cout <<setw(7)<<sal_a<<endl;
                unit_tot = unit_tot + sal_u;
                amt_tot = amt_tot + sal_a;
        }
```

```
void Sales::display()
	{
		cout <<"\t======================================"<<endl;
		cout << setw(15)<<"Total"
		<<setw(7)<<unit_tot<<setw(15)<<amt_tot<<endl;
		cout <<"\t======================================"<<endl;
	}

void main()
	{
		clrscr();
		Sales objS[3];
		int i;

		for (i=0;i<3;i++)
			objS[i].getdata();

		for (i=0;i<3;i++)
			objS[i].find_tot();

		Sales :: display();
		getch();
	}
```

- Draw the diagram
- How many member functions?
- How many non-static member functions?

Concept of New & Delete Operator

- In **C++**, the new and delete operators are used for **dynamic memory management**.
- They allow you to **allocate and de-allocate memory at runtime** efficiently and safely, replacing the older C functions malloc() and free().

1. The new operator:
    - Dynamically **allocates memory** for variables or objects **at runtime**.
    - Returns the **address** of the allocated memory.
    - **Automatically calls the constructor** (if it's a class object).
2. The delete operator:
    - **Frees the memory** that was previously allocated using new.
    - Prevents **memory leaks** by returning the memory back to the system.
    - **Automatically calls the destructor** (if it's a class object).

- Difference between New and Delete Operators

	New Operator	Delete Operator
Purpose	Allocates memory dynamically on the **heap** at runtime.	Deallocates (frees) dynamically allocated memory on the **heap**.
Syntax	pointer = new data_type; Example: int *p = new int;	delete pointer; Example: delete p;
For Arrays	pointer = new data_type[size]; Example: int *arr = new int[5];	delete[] pointer; Example: delete[] arr;
Constructor / Destructor Call	Automatically **calls constructor** (if allocating object).	Automatically **calls destructor** (if deleting object).
Returns	Returns the **address of allocated memory** (pointer).	Returns **nothing** (void).

Example:1 Use of New and Delete operators to allocate and De-allocate the memory

```
#include <iostream>

void main()
{
        int *arr,n;
        cout <<"How many elements are there?";
        cin>>n;
        *arr = new int[n];   // Dynamic allocation of Memory
         for (int i = 0; i < n; i++)
              {
                cout <<"Enter the Element of Array:";
                cin>>arr[i];
               }

        cout << "Array elements: ";
        for (int i = 0; i < n; i++)
             {
                cout << arr[i] << " ";
              }

   delete[] arr; // Free entire array

}
```

Example:2 Use of New and Delete operators to allocate and De-allocate the memory

```
#include <iostream>
#include <string.h>
class Student
{
Private:      char *name;
              int age;
public:
              Student(char *ptr, int a);
              Void display();
              ~Student();
};
  Student:: Student(char *ptr, int a)
{
              Cout<<"Constructor Called…"<<endl;
              Strcpy(name,ptr);
              age = a;
              cout << "Constructor called for " << name << endl;
}

Student:: ~Student()
{
      cout << "Destructor called for " << name << endl;
}

void Student::display()
{
              cout << "Name: " << name << ", Age: " << age << endl;
}
```

```
void main()
{
   // Dynamically create an array of objects
   Student *stud_ptr = new Student[2]{{"Ram", 23}, {"Sita", 21}};

   cout << "Array of Students:" << endl;
   for (int i = 0; i < 2; i++)
        {
              Stud_ptr[i].display();
         }

   delete[] stud_ptr;  // Calls destructors for all array objects
}
```

Example:3 Use of New and Delete operators to allocate and De-allocate the memory

```
#include <conio.h>
#include <iomanip.h>
#include <iostream.h>
#include <string.h>
class String
{
private:
              char *name;
              int length;
public:
              String()
              {
                    length=0;
                    name = new char[length+1];
              }
```

```
        String(char *s)
        {
                length=strlen(s);
                name = new char[length+1];
                strcpy(name,s);
        }
        void display()
        {
                cout << name<<endl;
        }
        void join(String &,String &);
};
void String::join(String &a, String &b)
{
        length = a.length + b.length;
        delete name;
        name = new char[length+1];
        strcpy(name, a.name);
        strcat(name, b.name);
}
void main()
{
        clrscr();
        String n1("MCA "),n2("Bhavnagar "),n3("University"),s1,s2;
        cout << "Size of n1 is : "<<sizeof(n1)<<endl;
        cout << "Size of n2 is : "<<sizeof(n2)<<endl;
        cout << "Size of n3 is : "<<sizeof(n3)<<endl;
        cout << "Size of s1 is : "<<sizeof(s1)<<endl;
        cout << "Size of s2 is : "<<sizeof(s2)<<endl;
        s1.join(n1, n2);
        s2.join(s1,n3);
        n1.display();
        n2.display();
        n3.display();
        s1.display();'
        s2.display();
}
```

Concept of Heap

- In **modern C++**, **smart pointers** are preferred for safe and automatic heap memory management.
- A **Heap** is a **complete binary tree** that follows a specific ordering property.
- The **heap** is a portion of RAM **managed by the operating system** and **allocated to the program**.
- It is used for **dynamic memory allocation**, meaning memory can be:
  - **Requested** during program execution.
  - **Released** when no longer needed.
- **Key Characteristics of Heap Memory**

Property	Explanation
Flexibility	Size can grow or shrink during program execution.
Lifetime	Memory persists until explicitly deallocated with delete.
Access	Only accessible through **pointers**.
Slower Access	Accessing heap memory is slower than stack memory because it involves pointers and OS-level management.
Management	The programmer must explicitly manage memory using new and delete.

- **How Heap Works Internally**

1. When you use new, the program sends a **request to the operating system** for a memory block.
2. The OS allocates the memory and returns the **starting address**.
3. The pointer stores this address and is used to access that memory.
4. When you call delete, the memory is **returned back to the OS**.
5. **Heap Memory:** Think of it like a **parking lot** with unlimited spots.
   You **park (allocate)** when you need space and **leave (delete)** when done.
6. **Dynamic Allocation:** The act of **parking a specific car in a spot** at runtime.
7. If you never leave (delete) the spot, other cars can't use it → **Memory Leak**.

Short note on Overloading

- Use of Same thing for different purpose or assign additional task to operator is known as Overloading.
- Function overloading is compiler time polymorphism implementing by multiple declaration of same function name with different types of parameters and number of arguments.
- In C++, two functions can have the same name if the number and/or type of arguments passed are different.
- Following functions having the same name but different arguments are known as overloaded functions.
- For example:

```
// same name different arguments
int test() { }
int test(int a) { }
float test(double a) { }
int test(int a, double b) { }
```

- Here, all 4 above functions are overloaded functions.
- Overloaded functions may or may not have different return types but they must have different arguments.
- For example,

```
// Error code
int test(int a) { }
double test(int b){ }
```

- Here, both functions have the same name, the same type, and the same number of arguments. Hence, the compiler will throw an error.

- Function overloading is similar to polymorphism that helps us to get different behaviour, with the same name of the function.
- Function overloading in C++ is used for code reusability and to save memory.
- Constructor member function overloading is possible because it may have different arguments. Relationship between class and Constructor member function is one-To-Many
- Destructor member function overloading is not possible because it must not have any arguments. Relationship between class and Destructor member function is One-To-One.
- Operator overloading is done by assigning special meaning to inbuilt operator whenever it is operated on objects.
- Operator overloading is a compile-time polymorphism. It is an idea of giving special meaning to an existing operator in C++ without changing its original meaning.
- Operator functions are the same as normal functions. The only differences are that the name of an operator function is always the operator keyword followed by the symbol of the operator and operator functions are called when the corresponding operator is used.
- Almost all operators can be overloaded except a few. Following is the list of operators that cannot be overloaded.

    1. Sizeof operator
    2. Scope resolution (::)
    3. Class member access operators (.(dot), .* (pointer to member operator))
    4. Ternary or conditional (?:)

- Operators that can be overloaded

1. Binary Arithmetic -> +, -, *, /, %
2. Unary Arithmetic -> +, -, ++, —
3. Assignment -> =, +=,*=, /=,-=, %=
4. Bit- wise -> & , | , << , >> , ~ , ^
5. De-referencing -> (->)
6. Dynamic memory allocation and De-allocation -> New, delete
7. Subscript -> [ ]
8. Function call -> ()
9. Logical -> &, | |, !
10. Relational -> >, < , = =, <=, >=

.

Example:01 Unary – operator overloading with TWO methods

```
#include <iostream.h>
#include <conio.h>
class Space
{
        private :       int x; float y;
        public  :
                        Space(int,float);
                        void  display();
                        void  operator - ();     // UNARY OPERATOR OVERLOADED
                        ~Space(){cout << "Destructor Called..."<<endl;}
};
Space::Space(int a,float b)
{
         cout << "Constructor Called..."<<endl;
         x = a;           y = b;
}
void Space::display()
{
         cout << "The value of X = "<<x<<endl;
         cout << "The value of Y = "<<y<<endl;
}
void Space::operator - ()
{
          x = -x;
          y = -y;
}
```

```
void main()
{
        clrscr();
        Space s(10,10.20);
        cout <<"Object S... "<<endl;
        s.display();
        -s;                    // NEGATIVE VALUE DISPLAY
        cout <<"Object S overloaded By Unary Operator...  "<<endl;
        s.display();
        cout <<"Object S overloaded By Unary Operator Second Method...  "<<endl;
        s.operator -();          // ORIGINAL VALUE DISPLAY
        s.display();
        getch();
}
```

Example:02 Binary operator * overloading with TWO methods

```
#include <iostream.h>
#include <conio.h>
class Space
{
        private :       int x;  float y;
        public  :       Space(int,float);
                        Space(){ cout << "Constructor Called..."; x=0; y=0;}
                         Space(Space &);
                        void  display();
                        Space operator * (Space); // BINARY OPERATOR
                        ~Space(){cout << "Destructor Called..."<<endl;}
  };
Space::Space(int a,float b)
{
         cout << "Constructor Called...";
         x = a;      y = b;
}
Space Space::operator *(Space c)
  {
         Space temp;
         temp.x = x * c.x;
        temp.y = y * c.y;
        return(temp);
  }
```

```
Space::Space(Space &ref)
{
        cout << "\t\tCopy Constructor Called..."<<endl;
        x = ref.x;      y = ref.y;
}
void Space::display()
{
         cout << "The value of X = "<<x;
       cout << "\t\tThe value of Y = "<<y<<endl;
}
void main()
{        clrscr();
         Space s1(20,20.40),s2(30,30.50),s3;
         cout <<endl<<"Object S1 Original Value...  "<<endl;
         s1.display();
         cout <<"Object S2 Original Value...  "<<endl;
         s2.display();
         cout <<"Object S3 overloaded By Binary Oprator...  "<<endl;
         s3 = s1 * s2;          // BOTH ARE SAME
         s3.display();
         cout <<"Object S3 overloaded By Binary Oprator Second Method...  "<<endl;
         s3 = s1.operator*(s2);  //S1 MEMBER FUNCTION CALL
         s3.display();
        getch();
}
```

Example:03 Unary -- operator overloading with TWO methods

```
#include <iostream.h>
#include <conio.h>
class Space
{       private :       int x;   int y;
        public  :       Space();
                        Space(int,int);
                        Space(Space &);
                        void  display();
                        Space operator -- ();      // UNARY OPERATOR OVERLOADED
                        ~Space() {cout << "Destructor Called..."<<endl;}
};
Space::Space()
{
        cout << "\constructor Called..."<<endl;
        x = 0; y = 0;
}
Space::Space(int a,int b)
{
         cout << "\constructor Called..."<<endl;
         x = a;   y = b;
 }
Space::Space(Space &ref)
{
        cout <<"\tCopy Constructor Called.."<<endl;
        x = ref.x;      y = ref.y;
 }
```

```
void Space::display()
  {
        cout << "The value of X = "<<x;
        cout << " \t\tThe value of Y = "<<y<<endl;
  }
Space Space::operator -- ()
{
         Space temp;
         temp.x = x--;  // temp.x = 2*x;
         temp.y = y--;
         return (temp);
}
void main()
 {
        clrscr();
        Space s(10,100),s1;  cout <<"Object S... "<<endl;
        s.display();
        s1 = --s;              // s1= s--;
        cout <<"Object S display original Value but y content is modified..."<<endl;
        s.display();
        cout <<"Object S1 overloaded By Unary Operator...  "<<endl;  s1.display();
        cout <<"Object S1 overloaded By Unary Operator Second Method...  "<<endl;
        s1 = s.operator --();
        s1.display();
        getch();
}
```

Example:04 Binary operator == (Equal to) overloading with TWO methods

```
#include <conio.h>
#include <iostream.h>
class Matrix
{       private:        int a[2][2];
        public :        Matrix();
                        int  operator == (Matrix);
                        Matrix(const Matrix &);
                        ~Matrix() {cout << "\t Destructor Called..."<<endl;}
};
void Matrix :: Matrix()
{       int i,j;
        cout << "Constructor Called Without Parameters..."<<endl;
        for (i=0;i<2;i++)
                for (j=0;j<2;j++)
                {       cout<< "Enter ["<<i<<"]["<<j<<"] =";
                        cin >>a[i][j];
                }
}
Matrix::Matrix (const Matrix & M)
 {
      int i,j;
      cout << "Copy Constructor Called..."<<endl;
        for (i=0;i<2;i++)
          for (j=0;j<2;j++)
                this->a[i][j] = M.a[i][j];
 }
```

```
int Matrix:: operator ==(Matrix m)
{       int i,j,flag=0;
        for (i=0;i<2;i++)
                for (j=0;j<2;j++)
                 {      if (a[i][j] == m.a[i][j])
                                flag=1;
                        else
                        {
                                flag=0;   break;
                        }
                }
         if (flag==1)   return(1);
         else         return(0);
 }
void main()
{       clrscr();
        cout<<"Enter The content of Matrix A..."<<endl;  Matrix obj1;
        cout<<"Enter The content of Matrix B..."<<endl;  Matrix obj2;
        cout <<" == Operator is overloaded..."<<endl;
        if (obj1 == obj2)  cout << "Matrix A and B are Equal"<<endl;
        else       cout << "Matrix A and B are not Equal"<<endl;
        cout <<" == Operator is overloaded by Second Method..."<<endl;
        if (obj1.operator == (obj2))
         cout << "Matrix A and B are Equal"<<endl;
        else
         cout << "Matrix A and B are not Equal"<<endl;
        getch();
}
```

Example:05 Unary operator++ overloading.

- Postfix and Prefix Operator Overloading.
- Postfix increment operator ++ is called with second argument 0

```
#include <conio.h>
#include <iostream.h>

class
{
        private :       int a;
         public  :      X(int i)  {cout << "Constructor With Parameter..."; a = i; }
                        X() { cout << "Constructor Called..."<<endl; }
                        X(const X &);                   // COPY CONSTRUCTOR
                         X operator++();                // PREFIX  OPERTOR OVERLOADING
                        X operator++(int);              // POSTFIX OPERTOR OVERLOADING
                        void display();                 // DISPLAY THE CONTENT OF a.
                        ~X(){cout<< "\t Destructor Called..."<<endl;}
 };

X::X(const X &x)
{
        cout << "\t\tCopy Constructor Called..."<<endl;
        this->a = x.a;    // a = =X.a;
}

X X::operator++()
{
         cout << "Prefix Increment Operator Called..."<<endl;
         X temp;    // DEFAULT CONSTRUCTOR CALLED
         temp.a = ++a;
         return(temp);
 }

X X::operator++(int)
 {
           cout << "Postfix Increment Operator Called..."<<endl;
           X temp;    // DEFAULT CONSTRUCTOR CALLED
           temp.a = a++;
           return(temp);
 }
```

```
void X::display()
{
        cout << "The Value of a: "<<a<<endl;
 }

void main()
{
        clrscr();
        X obj1(10),obj2(10),obj3;

        cout << "Content of Obj1 & Obj2 Without Overloading is..."<<endl;
        obj1.display();
        obj2.display();

        cout << "\ Content of Obj3 With Overloading is..."<<endl;
        obj3 = ++obj1;
        obj3.display();
        obj1.display();

        cout << "\n Content of Obj3 With Overloading is..."<<endl;
        obj3 = obj2++;
        obj3.display();
        obj2.display();
        getch();
  }
```

Example:06 Binary operator [] ➔ Subscript Operator and () ➔ Function Call operator overloading with **One Dimension Array**.

- The **subscript operator []** is used to **access elements** in arrays, pointers, and objects.
- For arrays: arr[i] is same as *(arr + i).
- For classes: Can be **overloaded** using operator [].
- The **subscript operator ([])** in C++ is considered a **binary operator**, even though it looks like it takes only one operand.
- When you write arr[i],
  it is interpreted internally as:

  *(arr + i)

- So it actually involves **two operands**:

  1. arr (the base address, usually a pointer or array)
  2. i (the index / offset)

- That's why the subscript operator is **binary**
- In C++, the **function call operator** () is actually considered a **binary operator** when we talk about operator overloading.
- When you write Obj(arguments) in our case V(location)
- obj(arg) is binary: one operand is the **object**, the other is the **argument(s)**.

```
#include<iostream.h>
#include<conio.h>
#define SIZE 10
class Array
{
        private :       int a[SIZE];
        public  :       Array();
                         int operator[](int index);
                        int operator()(int index);
                        ~Array() { cout << "Destructor Called..."<<endl;}
};
```

```
void Array :: Array()
{
	cout << "Constructor Called..."<<endl;
	for(int i=0; i<SIZE; i++)
	{
		cout<<" Enter ["<<i<<"] element : ";
		cin>>a[i];
	}
}
int Array :: operator[](int index)
{
	if(index >= SIZE || index < 0)
		return 9999;
	return a[index];
}

int Array :: operator()(int index)
{
	if(index >= SIZE || index < 0)
		return 9999;
	return a[index];
}
```

```
void main()
{
	clrscr();  Array v;    int location, temp;
	cout<<"\n Enter the location to search the value : ";
	cin>>location;
	temp = v[location];                    // temp = v.operator [](location);
	if(temp == 9999)
		cout<<"\n Index value is out of range.[from subscript operator]";
	else
		cout<<"\n Searched value from subscript operator is : "<<temp;
	//+++++++++++++++++++++++++++++++++++++++++++=

	temp = v(location);                          // temp = v.operator ()(location);
	if(temp == 9999)
		cout<<"\n Index value is out of range.[from function call operator]"<<endl;
	else
		cout<<"\n Searched value from function call operator is : "<<temp<<endl;
	getch();
}
```

Example:07 Binary operator [] ➔ Subscript Operator and () ➔ Function Call operator overloading with **Two Dimension Array**.

```
#include<iostream.h>
#include<conio.h>
#define ROW 3
 #define COL 3
class Array
{
        private :       int a[ROW][COL];
        public  :       Array();
                        int operator[](int index[2]);
                        int operator()(int []);
                        ~Array() {cout << "\nDestructor Called..."<<endl;}
};
Array :: Array()
{
        int i,j;
        cout << "Constructor Called..."<<endl;
        for(i=0; i<ROW; i++)
         {
                for(j=0; j<COL; j++)
                 {
                        cout<<" Enter ["<<i<<"]["<<j<<"] element : ";
                        cin>>a[i][j];
                 }
         }
}
```

```
int Array :: operator[](int index[2])
{       if( (index[0] >= ROW || index[0] < 0 ) || (index[1] >= COL || index[1] < 0 ) )
                return 9999;
        return a[index[0]][index[1]];
}
int Array :: operator()(int z[])
{       if( (z[0] >= ROW || z[0] < 0 ) || (z[1] >= COL || z[1] < 0 ) )
                return 9999;
        return a[z[0]][z[1]];
}
void main()
  {     clrscr();      Array v;       int  temp;    int i[2];
        cout<<"\n Enter the row to search the value : ";     cin>>i[0];
        cout<<"\n Enter the column to search the value : ";    cin>>i[1];
        temp = v[i];   //    temp = v.operator [](i);
        if(temp == 9999)
              cout<<"\n Index value is out of range.[from subscript operator]";
        else
              cout<<"\n Searched value from subscript operator is : "<<temp;
        //+++++++++++++++++++++++++++++++++++++++++++++
        temp = v(i);     // temp = v.operator ()(i)
        if(temp == 9999)
              cout<<"\n Index value is out of range.[from function call operator]";
        else
              cout<<"\n Searched value at ["<<i[0]<<"]["<<i[1]<<"] from function call
                          operator is : "<<temp;
        getch();
  }
```

Q.1 Implement the following in C++

- Create a class called Prog_1 and store a string
- Overload operator - (minus) to compare two stored strings. **Do not use strcmp()**
- Write appropriate code in main to test the above function
- Example: 1. Object1 – MBA, Object2 – MCA

  Display message: Object2 is greater than Object1

  2. Object1 – MCA, Object2 – MBA

  Display message: Object1 is greater than Object2

  3. Object1 – MCA, Object2 – MCA

  Display message: Object1 and Object2 are same
- Check for All types of input
- Class Prog_1

```
{
        Private:  Char str[50];
        Public :  Prog_1(char []);
                  Prog_1();
                  Prog_1(Prog_1 &);  // copy constructor
                  Prog_1 operator – (Prog_1);
                  Void display();
                  ~Prog_1();
}
```

Q.2 Write a C++ program to implement following

- Create a class Matrix of size 3 x 3. Write a program to overload the relational operator = = to check equality of two 3 x 3 matrices. Your program should support following main() :

```
void main()
{
   Matrix obj1,obj2 ;
    if(a = = b)
       cout << "Matrix A and B are equal" << endl ;
   else
       cout << "Matrix A and B are not equal" << endl ;
}
Class Matrix
    {       private: int a[3][3];
            Public : Matrix();
                     int operator == (Matrix);
                     Matrix(Matrix &);
                     ~Matrix();
     };
```

Q.3 Write a C++ program to implement following

- Create Class "Time" having two data members Hrs and Minutes
- Create two objects for class Time with the user of constructor
- Use operator overloading + to add the two objects
- Example:

Object – 1		Object - 2		Addition	
Hrs	Minute	Hrs	Minute	Hrs	Minute
5	25	3	55	9	20
3	45	5	25	9	10
6	40	5	45	12	25

Q.4 Write C++ program to implement following

- Create Class Space as follows

```
class Space
 {      private :  int x; float y;
        public  :  Space(int,float);
                   void  display();
                   void  operator - ();     // UNARY OPERATOR OVERLOADED
                   ~Space(){cout << "Destructor Called..."<<endl;}
 };
```

- Implement unary – operator overloading
- Write code for other member functions and main()

Q.5 Write a C++ program to implement following

- Create a class Matrix of size 2 x 2.
- Write a program to overload the binary operators + to add matrices.
- Your program should support following main() :

```
void main()
{
   Matrix a,b,c ;
   a.getdata() ; b.getdata() ;
   c=a+b ;
   c.putdata() ;
}
```

Q.6 Write a C++ program to perform the following:

- Declare class Liquid contains two members
  int liter, milliliter;
- Declare two objects L1 and L2 of class Liquid and pass above two parameters in the constructor.
- Declare object L of class Liquid and initialize it without parameter.
- Overload the Binary operator – and perform following
  L = L1 – L2;
- Display the output of with the use of display() function

Q.7 Write a C++ program to perform the following:

- Declare the class Matrix, having data member a[m][n]
- Allocate the memory of row and columns of Matrix with the use of new operator
- Overload postfix ++ operator and prefix ++ operator to increase value of each and every element of Matrix
- M = M1++
- M = ++M1 where M and M1 are the object of class Matrix
- Display the value of M and M1

Q.8 Write a C++ program to create a class matrix and declare integer 3X3 array as a data

- Use constructor to initialize data members.
- Declare a member functions to display a matrix.
- Declare an operator function '*=' to multiply two matrices and store result in first operand.
- Declare two objects matA and matB in main(), multiply these matrices as matA *= matB and display both the matrices.

Static data members and member functions

Q.1 Write a C++ program:

- Read the number from the terminal and print the digit by digit in words with the use of Static Data members and static member functions.
- Maximum limit is Ten Lakhs
- Examples:

Input		Output
1543	➔	One Thousand Five Hundred Forty Three Only
500	➔	Five Hundred Only
301	➔	Three Hundred One Only

```
//compare the string with overloading function and static member function
//Q1 without use of strcmp()

#include<iostream.h>
#include<iomanip.h>
#include<conio.h>
#include<string.h>
#include<stdio.h>

class prog_1
{
        private:        char str[50];
                        static int ans;
        public:         prog_1();
                       void  operator -(prog_1);
                       static void display();
                       ~prog_1(){cout<<"Destructer called"<<endl;}
};
int prog_1::ans=0;

prog_1::prog_1()
{
        cout<<"constructer called"<<endl;
        cout<<"Enter your string : ";  gets(str);
}

void prog_1::operator -(prog_1 c)
{
    int i; i=0;
    while(str[i]!=NULL || c.str[i]!=NULL)
            {
              if (str[i]==c.str[i])
                    {
                        i++;
                    }
            else if( str[i] < c.str[i])
                    {
                          ans=1; return;
                    }
            else if (str[i] > c.str[i])
                      {
```

```
                    ans=2; return;
                }
        }
}
void main()
{
    clrscr();
    prog_1 obj1,obj2;
    obj1-obj2;
    prog_1::display();
    getch();
}
void prog_1::display()
{
        if(ans==1)
            {
                cout<<"obj 2 is greater than obj 1"<<endl;
            }
    else if(ans==2)
            {
                cout<<"obj 1 is greater than obj 2 “<<endl;
            }
    else
            {   cout<<"obj 1 and obj 2 are same"<<endl;

            }
}
```

```
//Q3 overloading + with time class
#include<iostream.h> #include<conio.h>
#include<iomanip.h>
class time
{
        private:    int h,m;
        public:     time();
                    time(int,int);
                    time operator -(time &); void display();
                    ~time(){cout<<"destructor called"<<endl;}
};
time::time()
{
    cout<<"constructer called"<<endl; h=0;   m=0;
}
time::time(int hour,int min)
{
    cout<<"constructer called"<<endl; h=hour;       m=min;
}
time time::operator -(time &c)
{
       time temp;
       int t1_value,t2_value,t_value;
       t1_value=h*60 + m; t2_value=c.h*60+c.m;
       t_value=t1_value-t2_value; temp.h=t_value/60;
       temp.m=t_value%60;

       return temp;
}
void time::display()
{
    cout<<"hours:"<<h<<'\t'; cout<<"min: <<m<<endl;
}
void main()
{
    clrscr();
    time t1(0,53),t2(1,28),t3;
    t3=t1 - t2; t1.display();
    t2.display();  t3.display();
    getch();
}
```

```
//Q5 assignment
//overload the binary operators + add matrices

#include<iostream.h> #include<iomanip.h>
#include<conio.h>

class matrix
{
        private:        int a[2][2];
        public:         voidgetdata();
                        matrix operator + (matrix &);
                         void putdata();
};

void matrix::getdata()
{
    int i,j; for(i=0;i<2;i++)
            {
                for(j=0;j<2;j++)
                        {
                                cout<<"a["<<i<<"]["<<j<<"]=";
                                cin>>a[i][j];
                        }
            }
}

matrix matrix::operator +(matrix & c)
{
        matrix temp;
        int i,j;
        for(i=0;i<2;i++)
                {
                        for(j=0;j<2;j++)
                                {
                                    temp.a[i][j]=a[i][j]+c.a[i][j];
                                }
                }
                return temp;
}
```

```
void matrix::putdata()
{
    int i,j;
    for(i=0;i<2;i++)
        {
            for(j=0;j<2;j++)
                {
                    cout<<setw(4)<<a[i][j];
                }
          cout<<endl;
        }
}
void main()
{
    clrscr();
    matrix obj1,obj2,obj3;
    cout<<"matrixa..."<<endl;
    obj1.getdata();
    cout<<"matrixb..."<<endl;
     obj2.getdata();
    obj3 = obj1 + obj2 ;
    obj3.putdata();
    getch();
}
```

```
//postfix prefix operator overloading with matrix
#include<iostream.h> #include<iomanip.h>
#include<conio.h>

class matrix
    {

        private:  int a[3][3];
        public:   void getdata();
                  matrix(){cout<<"Constructor Called..."<<endl;}
                  matrix operator ++(int);  // Postfix notation
                  matrix operator ++();     // Prefix notation
                  void display();
                  ~matrix(){cout<<"Destructor called.."<<endl;}
    };

void matrix::getdata()
    {
       int i,j;
       for(i=0;i<3;i++)
            {
               for(j=0;j<3;j++)
                     {
                       cout<<"a["<<i<<"]["<<j<<"]=";
                       cin>>a[i][j];
                     }
            }
    }
matrix matrix::operator ++(int z)
    {
      matrix  temp;
       int i,j;
      for(i=0;i<3;i++)
            {
               for(j=0;j<3;j++)
                     {
                          temp.a[i][j]=a[i][j]++;
                     }
            }
          return (temp);
    }
```

```
matrix matrix::operator ++()
    {
       matrix  temp; int i,j;
       for(i=0;i<3;i++)
             {
                for(j=0;j<3;j++)
                      {
                            temp.a[i][j]=++a[i][j];
                      }
             }
           return (temp);
    }

void matrix::display()
    {
       int i,j;
       for(i=0;i<3;i++)
             {
                for(j=0;j<3;j++)
                    {
                            cout<<setw(4)<<a[i][j];
                     }
              cout<<endl;
             }
    }
void main()
    {
         clrscr();
         matrix m1,m;
         m1.getdata();
         cout<<"Postfix  Matrix  Overloading...:"<<endl;
         m=m1++;
         m.display();

         cout<<"Prefix Matrix overloading...:"<<endl;
         m=++m1;
         m.display();
         getch();
    }
```

```
// Matrix Multiplication
#include<iostream.h>
# include<iomanip.h>
#include<conio.h>
class matrix
    {
       private:     inta[3][3];
       public:      matrix()
                    { cout<<"Constructer called.."<<endl; }
                    void getdata();
                    matrix operator* (matrix&);
                    void display();
                    ~matrix(){cout<<"Destructor called..."<<endl;}
    };

void matrix::getdata()
    {
        int i,j;
       for( i=0;i<3;i++)
          {
             for( j=0;j<3;j++)
                {
                   cout<<"a["<<i<<"]["<<j<<"]=";
                   cin>>a[i][j];
                }
          }
    }

void matrix::display()
    {
       int i,j;
       for(i=0;i<3;i++)
          {
             for(j=0;j<3;j++)
                {
                   cout<<setw(4)<<a[i][j];
                }
             cout<<endl;
          }
    }
```

```
matrix  matrix::operator *(matrix & c)
    {
       matrixtemp;
       int i,j,k;

       for(i=0;i<3;i++)
          {
             for(j=0;j<3;j++)
                {
                   int sum=0; for(k=0;k<3;k++)
                      {
                         sum=sum + a[i][k] * c.a[k][j];
                      }
                   temp.a[i][j]=sum;
                }
          }
       return temp;
    }

void main()
    {
       clrscr();
       matrix obj1,obj2,obj3;
       cout<<"matrix a.."<<endl;
       obj1.getdata();

       cout<<"matrixb..."<<endl;
       obj2.getdata();

       obj3 = obj1 * obj2;  // obj3 = obj1.operator * (obj2)

       obj3.display();
       getch();
    }
```

Write a program to implement unary operator overloading on Matrix
Matrix having 3 rows and 4 columns

Input the value	After Unary operator overloading
12 66 77 35 -68 45 -99 101 88 52 64 81	-12 -66 -77 -35 68 -45 99 -101 -88 -52 -64 -81

```
#include<iostream.h>
#include<iomanip.h>
#include<conio.h>

class matrix
   {
        private:  int a[3][4];
         public:  matrix();                                          // Constructor
                  void operator -();                                 // Operator Overloading
                   void display();                                   // Normal Member function
                  ~matrix(){cout<<"Destructor called"<<endl;} // Destructor
   };

matrix::matrix()
   {
        int i,j;
        cout<<"constructer called..."<<endl;
        for(i=0;i<3;i++)
             {
                for(j=0;j<4;j++)
                   {
                       cout<<"enter["<<i<<"]["<<j<<"]=";
                        cin>>a[i][j];
                   }
             }
   }
```

```
void matrix::operator -()
   {
        int i,j;
        for(i=0;i<3;i++)
             {
               for(j=0;j<4;j++)
                  {
                     a[i][j]=-a[i][j];
                   }
             }
       }

void matrix::display()
   {
        int i,j;
        for(i=0;i<3;i++)
          {
             for(j=0;j<4;j++)
                {
                     cout<<setw(4)<<a[i][j];
                }
        cout<<endl;
        }
   }

void main()
   {
        clrscr();
        matrix obj;
        -obj;
        obj.display();
        cout<<"second method "<<endl;
        obj.operator -();
        obj.display();
        getch();
   }
```

Write a program to concate the two strings with the use of Operator overloading
Concate the string with the use of << operator and >> operator

Donot use strcat()

<table>
<tr><td colspan="4">Object s1</td><td colspan="4">Object s2</td></tr>
<tr><td>M</td><td>C</td><td>A</td><td>\0</td><td>M</td><td>B</td><td>A</td><td>\0</td></tr>
<tr><td colspan="4">Left➔ Right Extraction Operator Overloading >> (Concate from S1 into s2)</td><td colspan="4">Right➔ Left Insertion Operator Overloading << (Concate from s2 into s1)</td></tr>
<tr><td>M</td><td>B</td><td>A</td><td></td><td>M</td><td>C</td><td>A</td><td>\0</td><td>M</td><td>C</td><td>A</td><td></td><td>M</td><td>B</td><td>A</td><td>\0</td></tr>
<tr><td colspan="8">Object s2</td><td colspan="8">Object s1</td></tr>
</table>

```
#include<iostream.h>
#include<iomanip.h>
#include<conio.h>
#include<string.h>
#include<stdio.h>
class string
{
   private: char str[50];
   public:  string();
            void operator <<(string &);
            void operator >>(string &);
            void display();
            ~string(){cout<<"destructer called..."<<endl;}
};
string::string()
{
  cout<<"constructer called...:"<<endl;
  cout <<"Str:";    gets(str);
}
void string::operator <<(string &c)
{
    int n,i;
    n=strlen(str);
    i=0;
    str[n++]=' ';
    while(c.str[i]!=NULL)
         {
              str[n]=c.str[i];
              n++;
              i++;
         }
         str[n]=NULL;
}
```

```
void string::operator >>(string &c)
  {
      int n,i;
      n=strlen(c.str);
      i=0;
      c.str[n++]=' ';
      while(str[i]!=NULL)
         {
           c.str[n]=str[i];
           n++;
           i++;
         }
         c.str[n]=NULL;
  }
  void string::display()
  {
      cout<<"join string1 & string2"<<endl;
      puts(str);
  }
  void main()
  {
     clrscr();    string s1,s2;   int choice;
     cout << "Enter 1-String Concate from Left==>Right"<<endl;
     cout << "Enter 2-String Concate from Right==>Left"<<endl;
     cin>>choice;
     switch(choice)
        {
         case 1:
          {
               cout<<"strcat Left==>Right:"<<endl;
               s1>>s2;   // String concate from s1==>s2
               s2.display();           break;
           }
         case 2:
          {
              cout<<"strcat Right ==> Left"<<endl;
              s1<<s2;    // String Concate from s2==>s1
              s1.display();      break;
          }
        }
      getch();
  }
```

Write a program to display truth table for 3-inputs

```
#include<iostream.h>
#include<iomanip.h>
#include<conio.h>

class gate
{
  private:  static int value[8][3];
  public:   void and_op();
            void or_op();
            void ex_op();
            ~gate(){cout<<"Destructor called..."<<endl;}
};
int gate::value[8][3]={{0,0,0},{0,0,1},{0,1,0},{0,1,1},{1,0,0},{1,0,1},{1,1,0},{1,1,1}};

void gate::and_op()
{
  int i,ans;
  for(i=0;i<8;i++)
    {
        if(value[i][0] || value[i][1] || value[i][2])
            ans=1;
        else
            ans=0;
        cout<<setw(4)<<value[i][0]<<setw(4)<<value[i][1]<<setw(4)<<value[i][2];
        cout<<"==>"<<setw(4)<<ans<<endl;
   }
}
void gate::or_op()
{
     int i,ans;
     for(i=0;i<8;i++)
       {
        if(value[i][0] && value[i][1] && value[i][2])
                    ans=1;
        else
                     ans=0;
        cout<<setw(4)<<value[i][0]<<setw(4)<<value[i][1]<<setw(4)<<value[i][2];
        cout<<"==>"<<setw(4)<<ans<<endl;
     }
}
```

```
void gate::ex_op()
{
      int i,ans;
      for(i=0;i<8;i++)
            {
                if(value[i][0] ^ value[i][1] ^ value[i][2])
                        ans=1;
                 else
                         ans=0;
              cout<<setw(4)<<value[i][0]<<setw(4)<<value[i][1]<<setw(4)<<value[i][2];
              cout<<"==>"<<setw(4)<<ans<<endl;
               }
}
void main()
{
     clrscr();
     gate table1,table2,table3;
     cout <<" Truth table for AND Gate"<<endl;
     table1.and_op();
     getch();

     cout <<" Truth Table for OR Gate"<<endl;
     table2.or_op();
     getch();

     cout <<" Truth table for Exclusive OR gate"<<endl;
     table3.ex_op();
     getch();
}
```